

ShelfLife — Complete Project Guide & Interview Handbook

A from-scratch, plain-English walkthrough of the entire ShelfLife backend (with the React frontend summarized). Written for someone brand-new to the project who needs to understand **everything** — architecture, every class, every endpoint, the database, all cross-cutting concerns — and walk into an interview confident. Read top to bottom the first time. After that, use the Table of Contents to jump around.

Table of Contents

1. [What ShelfLife Is \(the 2-minute version\)](#)
 2. [The Big Picture: Architecture & Diagrams](#)
 3. [Folder & Package Structure](#)
 4. [End-to-End Request Flow](#)
 5. [Database Schema \(every table, column, relationship, index\)](#)
 6. [Build, Configuration & Cross-Cutting Infrastructure](#)
 7. [Security & Authentication \(JWT + RBAC\)](#)
 8. [Users, Warehouses & Data Isolation](#)
 9. [Inventory: Products & Shelf Locations](#)
 10. [Order Ingestion \(CSV / Excel Upload\)](#)
 11. [Picker Workflow & Scan-to-Pick](#)
 12. [Audit Trail & Dispatch Reporting](#)
 13. [The Frontend \(React SPA\)](#)
 14. [Design Patterns Used \(catalogue\)](#)
 15. [Interview Questions & Answers \(basic → advanced\)](#)
 16. [Improvements & Best Practices](#)
-

1. What ShelfLife Is

ShelfLife is the backend "brain" of a warehouse order-picking system. Imagine a big distribution center: orders come in (often as spreadsheets), each order is a list of products to collect, and a warehouse worker (a **picker**) walks the aisles with a handheld scanner, scanning each item until the order is complete. ShelfLife manages all of that.

It is built as a **Spring Boot 3 / Java 21 REST API** with a **PostgreSQL** database and a **React** single-page app on top. There are two kinds of users:

Role	Who they are	What they can do
CENTRAL_ADMIN	Head-office administrator	Everything, across all warehouses: manage users, warehouses, products, shelf maps, upload orders, view reports.
HUB_PICKER	Warehouse-floor worker	Only their assigned warehouse(s): pick orders, scan items.

The product is organized around five real-world capabilities (called BRD sections — BRD = *Business Requirements Document*):

- **3.1 Secure login + data isolation** — log in, and pickers only ever see their own warehouses.
- **3.2 Master data + order upload + routing** — products, shelf locations, bulk order upload, and the optimal walking route through the aisles.
- **3.3 Picker workflow + scan-to-pick** — claim an order, scan items one by one.
- **3.4 Dispatch report** — management reporting on what was picked vs ordered.
- **§4 Audit trail** — an immutable log of every important action.

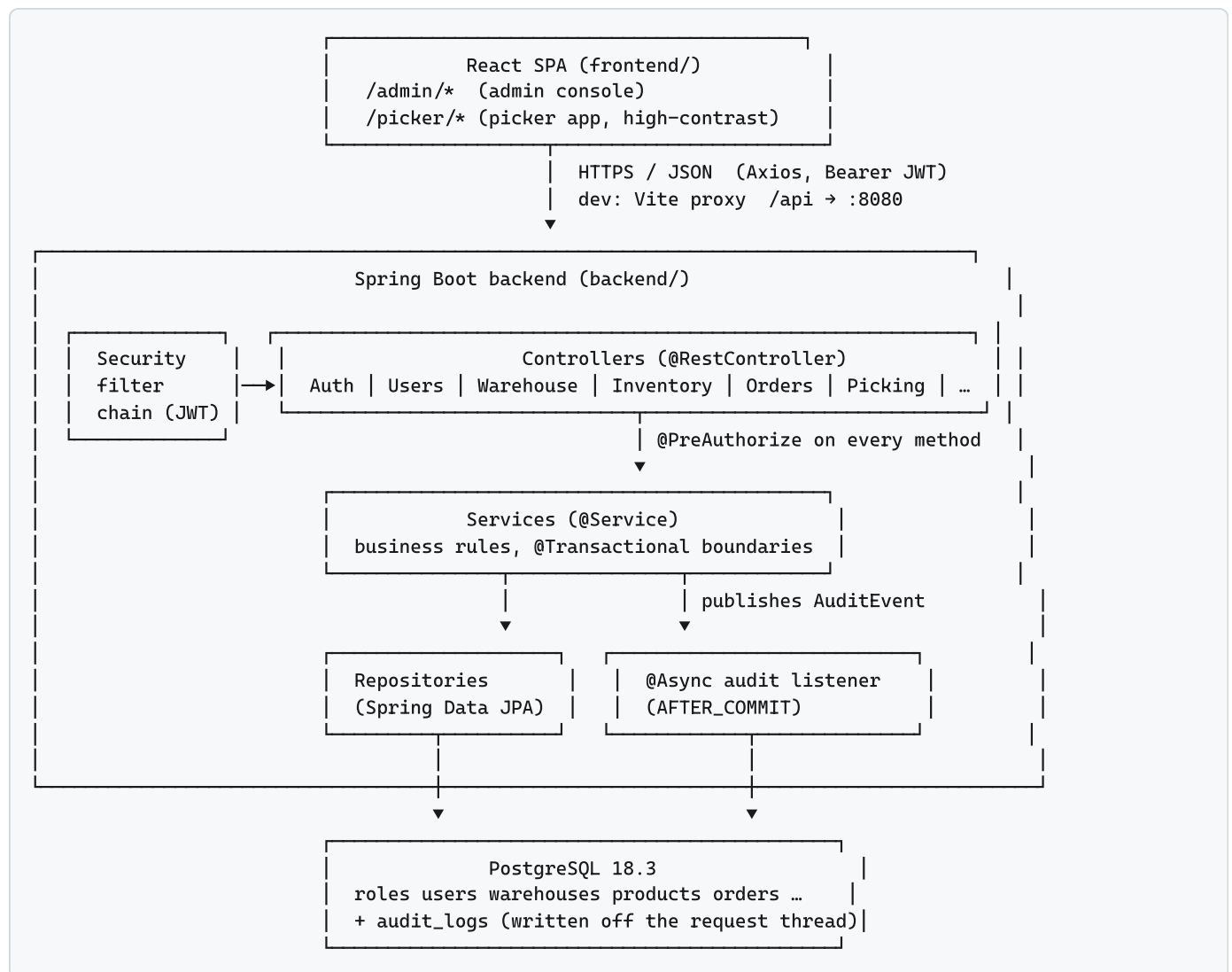
All of these are **implemented**.

2. Architecture & Diagrams

2.1 Technology stack

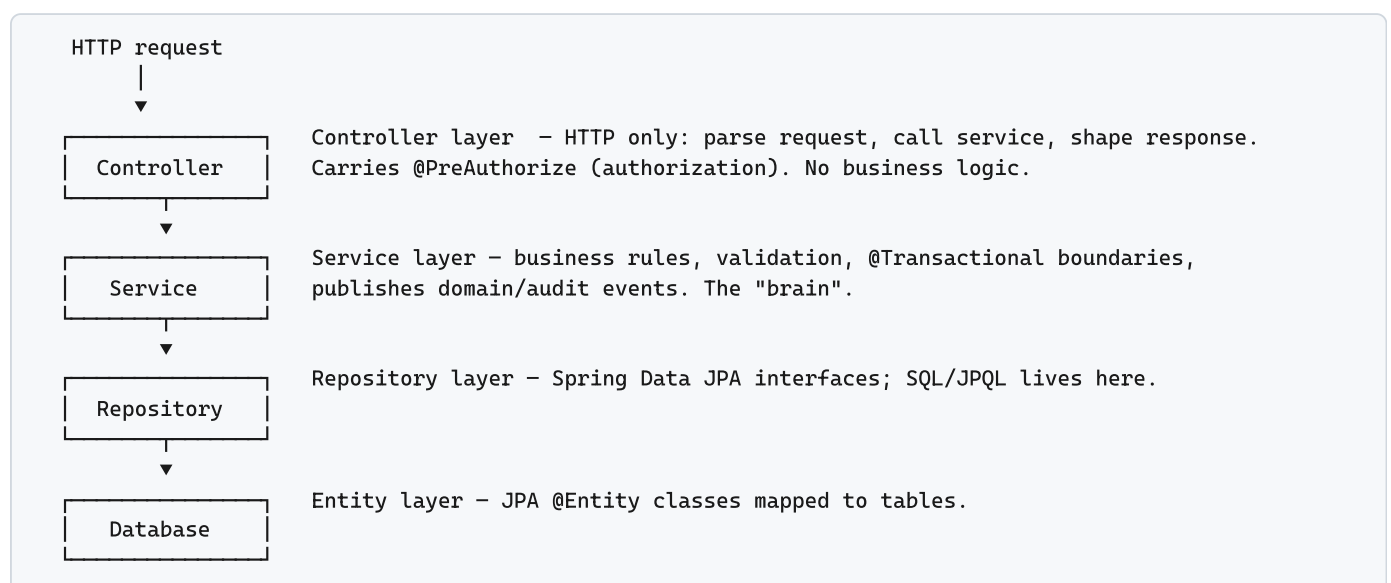
Layer	Technology
Language/Runtime	Java 21 (LTS)
Framework	Spring Boot 3.5.3 (Spring MVC, Spring Security, Spring Data JPA)
Persistence	Hibernate (JPA) + Spring Data repositories
Database	PostgreSQL 18.3
Auth	Stateless JWT (HS256 access tokens) + hashed opaque refresh tokens
File parsing	OpenCSV (CSV), Apache POI (Excel)
Mapping/boilerplate	MapStruct + Lombok
API docs	springdoc-openapi (Swagger UI)
Build	Maven
Frontend	Vite + React 18 + TypeScript + Material UI v5 + React Query + Axios
Packaging	Docker Compose (postgres + backend + frontend)

2.2 Component diagram (high level)



2.3 Layered architecture

ShelfLife follows the classic **layered (n-tier) architecture**, organized **package-per-feature**:



Why this layering? Each layer has one job, so the code is testable (you can test a service without HTTP), swappable (change the DB without touching controllers), and readable. DTOs cross the controller boundary so the API contract is decoupled from the database tables.

3. Folder & Package Structure

3.1 Repository layout (monorepo)

```
ShelfLife/
├── backend/           Spring Boot app (run mvn from here)
│   ├── pom.xml
│   └── src/main/
│       ├── java/com/ashu/shelflife/  ← all Java code (see package table below)
│       └── resources/
│           ├── application.yml        ← config
│           └── db/schema.sql          ← the whole schema + seed data (idempotent)
├── frontend/         Single Vite + React + TS + MUI SPA (admin + picker)
├── .claude/           Project docs / design notes
└── docker-compose.yml postgres + backend + frontend
```

3.2 Java package-per-feature layout

backend/src/main/java/com/ashu/shelflife/

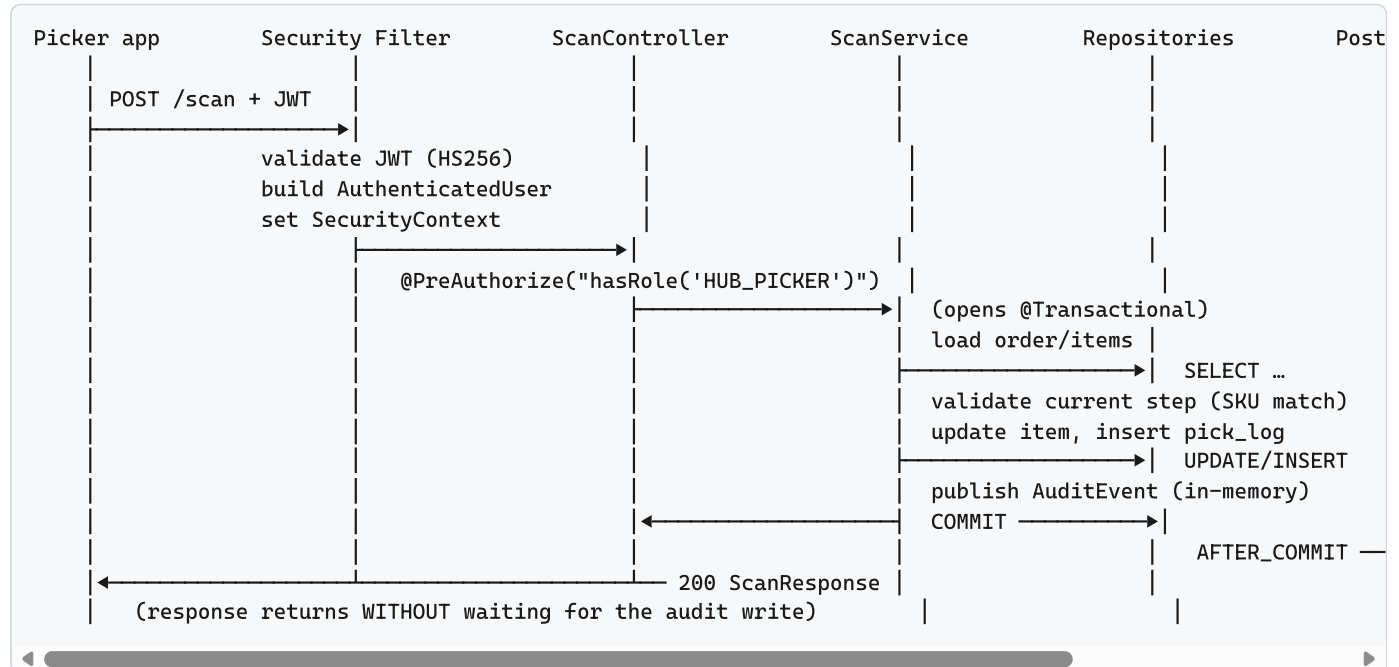
Package	Responsibility
config	App-wide configuration (async executor, OpenAPI/Swagger).
security	JWT issuing/verifying, the auth filter, RBAC handlers, BCrypt encoder.
auth	Login, refresh, logout; refresh-token persistence.
users	Users + roles CRUD.
warehouse	Warehouses, picker↔warehouse mappings, and WarehouseScope (isolation).
inventory	Products + shelf locations (master data).
orders	Orders + order items + CSV/Excel ingestion.
picking	Active warehouse, dashboard, claim, routing, scan-to-pick, sessions/logs.
reports	Dispatch report (JSON/CSV/Excel).
audit	Audit event + async listener + audit_logs.
common	Shared error types (ApiError , exceptions, global handler).

Why package-per-feature (instead of package-by-layer)? Everything about "orders" lives in one folder — entity, repo, service, controller, DTOs. You can reason about a feature without hunting across controllers/ , services/ , repositories/ folders. It scales better as the app grows and makes ownership obvious.

4. End-to-End Request Flow

Here is what happens for a typical authenticated call — say a picker scanning an item (`POST /scan`). This is the single most important mental model for the whole app.

4.1 Sequence diagram



4.2 Step-by-step in words

1. **HTTP arrives.** The browser sends `POST /scan` with header `Authorization: Bearer <jwt>`.
2. **Security filter (`JwtAuthenticationFilter`).** Before the request reaches any controller, the filter extracts the token, verifies its signature & expiry with `JwtService`, builds an `AuthenticatedUser` principal (user id, role, email, warehouse ids), and stores it in the Spring `SecurityContext`. No token / bad token → 401 via `RestAuthenticationEntryPoint`.
3. **Authorization (`@PreAuthorize`).** Spring method security checks the rule on the controller method (e.g. `hasRole('HUB_PICKER')`). Wrong role → 403 via `RestAccessDeniedHandler`.
4. **Controller.** Thin: it deserializes the JSON body into a request DTO (Bean Validation runs on `@Valid`), then calls the service.
5. **Service (`@Transactional`).** A database transaction opens. The service runs business rules: it checks the picker owns the order, that the scanned SKU matches the current route step, then updates the item, inserts a `pick_logs` row, and maybe completes the order. It **publishes an `AuditEvent`** (just an in-memory event — no DB write yet).
6. **Repositories.** Spring Data JPA translates method calls / `@Query` into SQL against Postgres.
7. **Commit.** The transaction commits. Only *now* does the audit listener fire (`@TransactionalEventListener(AFTER_COMMIT)` + `@Async`), writing the audit row on a **separate thread** so the response is not delayed.
8. **Response.** The controller serializes a response DTO to JSON → 200 OK. If anything threw, the `GlobalExceptionHandler` converts it into a structured `ApiError` JSON with the right status.

4.3 DTO vs Entity (why two object types)

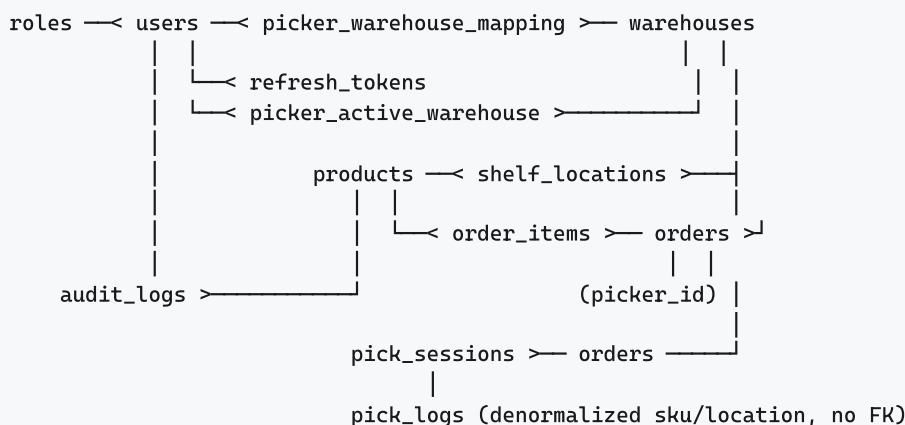
- **Entity** (e.g. `Order`) = mapped to a DB table, lives in transactions, may have lazy relationships. Never exposed directly to the web.
- **DTO** (e.g. `ClaimResponse`, `CreateUserRequest`) = the API's public shape. Keeps the database free to change, prevents accidentally leaking fields (like `password_hash`), and is where Bean Validation annotations live.

5. Database Schema

PostgreSQL 18.3, database `shelflife`. The **entire** schema plus demo seed data lives in one **idempotent** file: `backend/src/main/resources/db/schema.sql`. Spring runs it on **every** startup (`spring.sql.init.mode=always`), *before* Hibernate validates the mappings. Idempotent means every statement is safe to re-run: `CREATE TABLE/INDEX IF NOT EXISTS` and seed `INSERT ... ON CONFLICT DO NOTHING`.

Important design rule: the schema is **owned by SQL**, and Hibernate is set to `ddl-auto: validate` — it may *check* that entities match the tables but must **never** create or alter them. All schema changes go through the SQL file. (Historical Flyway migrations `V1...V5` are retained for reference but Flyway is disabled; `schema.sql` is the consolidated truth.)

5.1 Entity-Relationship diagram



—< means "one-to-many" (the < side is the many).

5.2 Table-by-table

roles

`id` (PK) · `name` (UNIQUE). Seeded with `CENTRAL_ADMIN`, `HUB_PICKER`. The whole RBAC system hangs off these two rows.

users

`id` PK · `name` · `email` (UNIQUE — login key) · `password_hash` (BCrypt) · `role_id` (FK→roles) · `status` · `created_at`.

- `status` is `VARCHAR + CHECK IN ('ACTIVE', 'INACTIVE', 'SUSPENDED')`, default `ACTIVE`.

- Index on `role_id`.
- **Why `status` as `VARCHAR+CHECK` and not a native PG enum?** It maps directly to a Java enum via JPA `@Enumerated(String)`, and adding a new status is just editing the CHECK — no fragile `ALTER TYPE`. It's also portable across databases.

`refresh_tokens` (auth)

`id` PK · `user_id` (FK→users) · `token_hash` (UNIQUE, 64-char SHA-256 hex) · `expires_at` · `revoked` (default false) · `created_at`. Index on `user_id`.

- **Only the SHA-256 hash** of the opaque refresh token is stored — if the DB leaks, the actual tokens aren't usable. Supports logout (set `revoked=true`) and rotation (revoke old, insert new).

`warehouses`

`id` PK · `warehouse_code` (UNIQUE — the human business key, e.g. WH-MAIN) · `warehouse_name` · `address`.

`picker_warehouse_mapping` (which pickers can work which warehouses)

Composite PK (`picker_id`, `warehouse_id`), both FKs. Index on `warehouse_id`. This is the heart of **data isolation**: a picker may only touch warehouses listed here for them.

`picker_active_warehouse` (the picker's "currently selected" warehouse)

`picker_id` PK (FK→users) · `warehouse_id` (FK→warehouses) · `selected_at`.

- **Why a table and not session state?** Auth is *stateless* (JWT carries no server session), so "which warehouse am I working in right now" must be persisted somewhere. One row per picker.

`products`

`sku` PK (**VARCHAR, natural key**) · `name` · `barcode` · `unit`. Index on `barcode`.

- **Why `sku` is the primary key (not a surrogate id):** SKU is the real business identifier and is referenced directly by `shelf_locations`, `order_items`, and `pick_logs`. Every other table uses a surrogate `BIGINT IDENTITY`.

`shelf_locations` (where each product lives in each warehouse)

`id` PK · `warehouse_id` (FK) · `sku` (FK) · `aisle` · `bay` · `shelf` · `location_code`.

- **UNIQUE (`warehouse_id`, `sku`)** — a product has exactly one shelf per warehouse.
- `location_code` example: Aisle_A-Bay_04-Shelf_2. Index on `sku`.

`orders`

`id` PK · `order_number` (UNIQUE) · `customer_id` (VARCHAR(64)) · `warehouse_id` (FK) · `picker_id` (FK→users, nullable — set when claimed) · `status` · `version` (INT default 0) · `created_at`.

- `status` CHECK IN ('PENDING', 'ASSIGNED', 'PICKING', 'COMPLETED', 'DISPATCHED'), default `PENDING`.
- `version` is the **optimistic-lock counter** used by the claim operation.
- Indexes on `warehouse_id`, `status`, `picker_id`.

order_items (the lines of an order)

`id` PK · `order_id` (FK) · `sku` (FK) · `item_name` · `ordered_quantity` · `picked_quantity` (default 0) · `status` CHECK IN ('PENDING', 'PICKED', 'SKIPPED') default PENDING. Indexes on `order_id`, `sku`.

pick_sessions (one per claimed-and-worked order)

`id` PK · `order_id` (FK) · `picker_id` (FK) · `start_time` · `end_time` · `status` CHECK IN ('OPEN', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED') default OPEN. Indexes on `order_id`, `picker_id`.

pick_logs (an immutable record of each scan)

`id` PK · `session_id` (FK) · `sku` · `quantity` · `location` · `scan_time` · `scan_id` (UNIQUE, nullable). Index on `session_id`.

- **`sku` / `location` are denormalized snapshots with no FK on purpose** — logs are append-only history; they must survive even if a product or shelf is later renamed or deleted.
- **`scan_id`** is the optional idempotency key (UNIQUE; NULLs are distinct in Postgres so keyless scans still work). Prevents double-counting when a scanner retries.

audit_logs (the \$4 audit trail)

`id` PK · `user_id` (FK) · `action` · `order_id` (nullable) · `sku` (nullable) · `location` (nullable) · `timestamp`. Indexes on `user_id`, `order_id`.

5.3 Seed data (what you get on a fresh DB)

- Roles `CENTRAL_ADMIN`, `HUB_PICKER`.
- Demo users: `admin@shelflife.com` / `Admin@123` and `picker@shelflife.com` / `Picker@123` (passwords stored BCrypt-hashed).
- Two warehouses (`WH-MAIN`, `WH-NORTH`), the picker mapped to both.
- Five products and their `WH-MAIN` shelf locations (codes pre-sorted into route order for a tidy picking demo).

Seeds use natural keys (`roles.name`, `users.email`, `warehouse_code`, `sku`) in `SELECT ... FROM` sub-queries with `ON CONFLICT DO NOTHING`, so they are independent of whatever `IDENTITY` values get assigned and never duplicate on re-run.

6. Build, Configuration & Cross-Cutting Infrastructure

This section covers everything that wires the application together but isn't a business feature: the Maven build, runtime configuration, container orchestration, the application bootstrap class, the cross-cutting `@Configuration` beans (async executor + OpenAPI), and the global error-handling layer.

6.1 pom.xml — Build & Dependencies

The project inherits from the Spring Boot **parent BOM**, which is what lets most dependencies omit `<version>` — versions are managed centrally:


```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.3</version>
  <relativePath/>
</parent>

<groupId>com.ashu</groupId>
<artifactId>shelflife</artifactId>
<version>0.0.1-SNAPSHOT</version>
```

- `spring-boot-starter-parent` supplies dependency management (a curated, mutually-compatible set of library versions) and sensible plugin defaults. Inheriting it is why `spring-boot-starter-web`, `-security`, `-data-jpa`, etc. carry no explicit version below.
- `<relativePath/>` (empty) tells Maven **not** to look for the parent POM on disk relative to this file; it resolves the parent from the repository instead.

Version properties pin libraries **not** managed by the Boot BOM:

```
<properties>
  <java.version>21</java.version>
  <mapstruct.version>1.6.3</mapstruct.version>
  <lombok-mapstruct-binding.version>0.2.0</lombok-mapstruct-binding.version>
  <poi.version>5.4.0</poi.version>
  <opencsv.version>5.10</opencsv.version>
  <jjwt.version>0.12.6</jjwt.version>
  <springdoc.version>2.8.9</springdoc.version>
</properties>
```

Key dependencies — what & why

Dependency	Purpose / why it's here
<code>spring-boot-starter-web</code>	Spring MVC + embedded Tomcat — the REST controllers and HTTP server.
<code>spring-boot-starter-security</code>	Authentication/authorization plumbing — backs the JWT filter chain, <code>@PreAuthorize</code> method security, password encoding.
<code>spring-boot-starter-data-jpa</code>	JPA + Hibernate + connection pooling (HikariCP) for the entity/repository layer.
<code>spring-boot-starter-validation</code>	Jakarta Bean Validation (Hibernate Validator) — powers <code>@Valid</code> / <code>@NotNull</code> on request DTOs.
<code>org.postgresql:postgresql (runtime)</code>	The PostgreSQL JDBC driver. <code>runtime</code> scope: needed to run/test, not to compile against.
<code>flyway-core + flyway-database-postgresql</code>	Flyway migration engine. Present but disabled at runtime ; schema ownership moved to <code>db/schema.sql</code> .
<code>org.projectlombok:lombok (optional)</code>	Compile-time boilerplate (getters, builders, constructors). <code>optional=true</code> so it doesn't leak transitively.
<code>org.mapstruct:mapstruct</code>	Compile-time DTO↔entity mapper generation.
<code>org.apache.poi:poi + poi-ooxml</code>	Excel (<code>.xlsx</code>) generation (report) and reading uploaded Excel order files.
<code>com.opencsv:opencsv</code>	CSV parsing (order upload) and CSV writing (report export).
<code>io.jsonwebtoken:jjwt-api (compile) + jjwt-impl (runtime) + jjwt-jackson (runtime)</code>	The JWT library split into stable API (compile) + impl + Jackson serializer (runtime). App code depends only on the API.
<code>org.springdoc:springdoc-openapi-starter-webmvc-ui</code>	Auto-generates the OpenAPI 3 spec and serves Swagger UI.
<code>spring-boot-starter-test (test)</code>	JUnit 5, Mockito, AssertJ, Spring test context.
<code>spring-security-test (test)</code>	Security-aware test helpers.

Build plugins. `spring-boot-maven-plugin` repackages into an executable fat JAR and excludes Lombok from the artifact (it's build-time-only). The most interview-relevant block is the **annotation-processor pipeline** in `maven-compiler-plugin`, where *ordering matters*:

```
<annotationProcessorPaths>
  <path><groupId>org.projectlombok</groupId><artifactId>lombok</artifactId> ... </path>
  <path><groupId>org.mapstruct</groupId><artifactId>mapstruct-processor</artifactId> ... </path>
  <path><groupId>org.projectlombok</groupId><artifactId>lombok-mapstruct-binding</artifactId> ... </path>
</annotationProcessorPaths>
<compilerArgs>
  <arg>-Amapstruct.defaultComponentModel=spring</arg>
</compilerArgs>
```

- Three processors run at compile time: **Lombok** (generates accessors), **mapstruct-processor** (generates mapper impls), and **lombok-mapstruct-binding** — the critical glue ensuring MapStruct runs *after* Lombok so it can "see" the Lombok-generated getters/setters (without it, MapStruct may run first and generate broken mappers).
- `-Amapstruct.defaultComponentModel=spring` makes every generated mapper a Spring `@Component` (injectable bean), instead of the default static `INSTANCE`.

6.2 application.yml — Runtime Configuration

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/shelflife
    username: postgres
    password: "Ashu1234@"
  jpa:
    hibernate:
      ddl-auto: validate
    defer-datasource-initialization: false
    open-in-view: false
  sql:
    init:
      mode: always
      schema-locations: classpath:db/schema.sql
  flyway:
    enabled: false
security:
  jwt:
    secret: ${SECURITY_JWT_SECRET:c2hlbGZsaWZLL ... base64 ... }
    access-token-expiration-minutes: 15
    refresh-token-expiration-days: 7
    issuer: shelflife
server:
  port: 8080
```

- **ddl-auto: validate** — Hibernate **never creates/alters** tables; on startup it only validates entity mappings against the existing schema. The safe production-grade setting.
- **defer-datasource-initialization: false** — deliberately false so the `schema.sql` init script runs **before** the JPA `EntityManagerFactory` is built. Tables must exist before Hibernate's `validate` inspects them. (If `true`, `validate` would fail on a fresh DB.)
- **open-in-view: false** — disables the Open-Session-In-View anti-pattern; the persistence session does not stay open for the whole request, forcing lazy loading to happen inside service transactions. Good practice.
- **sql.init.mode: always** — runs `classpath:db/schema.sql` on every boot. Safe because the script is idempotent.
- **flyway.enabled: false** — schema management handed to `schema.sql`; the `V1..V5` scripts are legacy reference.
- **security.jwt.secret** — Base64-encoded HMAC-SHA256 key (≥ 256 bits) via `${ENV:default}`: read `SECURITY_JWT_SECRET` from env, fall back to the embedded dev-only key. Real deployments override it.
- Access tokens live 15 min; refresh tokens 7 days; iss claim = `shelflife`. Server on 8080.

6.3 docker-compose.yml — Full-Stack Orchestration

Three services bring up a pre-seeded stack with one `docker compose up`.

- **postgres** — `postgres:16-alpine` (note: containerized PG is **16**, not the local-machine 18.3). DB/user/password all `shelflife`. Named volume `shelflife-pgdata` persists data (`down -v` wipes for a clean reset). Published on host port **55432:5432** to avoid clashing with local PostgreSQL. Healthcheck uses `pg_isready`.
- **backend** — built from `./backend`. `depends_on: postgres: condition: service_healthy` (waits for DB *healthy*, not just started). `SPRING_DATASOURCE_*` env vars override the localhost defaults; the host is the Docker service name `postgres` on internal port `5432`. Healthcheck hits the whitelisted `/v3/api-docs`; `start_period: 40s` accounts for JVM + Spring + schema init.
- **frontend** — built from `./frontend` (React served by nginx, proxying `/api` → backend). Waits for backend healthy. Publishes host `3000` → container `80`.

Dependency chain: postgres (healthy) → backend (healthy) → frontend.

6.4 Bootstrap & @Configuration Classes

ShelflifeApplication.java — the entry point:

```
@SpringBootApplication
public class ShelflifeApplication {
    public static void main(String[] args) {
        SpringApplication.run(ShelflifeApplication.class, args);
    }
}
```

@SpringBootApplication is a meta-annotation bundling **@SpringBootConfiguration** + **@EnableAutoConfiguration** (Boot wires beans from the classpath — Tomcat, JPA, Security) + **@ComponentScan** (scans `com.ashu.shelflife` and below). Sitting at the package root means scanning covers all feature packages.

AsyncConfig.java — the audit executor (the engine behind off-thread auditing):

```
@Configuration
@EnableAsync
public class AsyncConfig {
    @Bean(name = "auditExecutor")
    public Executor auditExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(2);
        executor.setMaxPoolSize(4);
        executor.setQueueCapacity(1000);
        executor.setThreadNamePrefix("audit-");
        executor.initialize();
        return executor;
    }
}
```

- **@EnableAsync** activates Spring's **@Async** proxying. Without it, **@Async** is silently ignored.
- **@Bean(name = "auditExecutor")** — a named pool referenced by **@Async("auditExecutor")**. `corePoolSize=2`, `maxPoolSize=4`, `queueCapacity=1000` (large buffer; audit events are bursty), `threadNamePrefix="audit-"` for log readability. Interview nuance: with a 1000-deep queue, the pool stays at 2 threads until the queue fills, *then* grows to 4 (standard `ThreadPoolExecutor` semantics).

OpenApiConfig.java — sets the Swagger title/description and registers an HTTP **Bearer JWT** security scheme globally (`Type.HTTP`, `scheme("bearer")`, `bearerFormat("JWT")`), so the Swagger UI **Authorize** button accepts a raw `accessToken` from `/auth/login` and sends it as `Authorization: Bearer <token>`. The docs paths are whitelisted in **SecurityConfig**.

6.5 common/error — Global Error Handling

A uniform error contract: every handled failure returns the same JSON shape.

ApiError — the error body, a Java `record`:

```

public record ApiError(OffsetDateTime timestamp, int status, String error, String message, String path) {
    public static ApiError of(int status, String error, String message, String path) {
        return new ApiError(OffsetDateTime.now(), status, error, message, path);
    }
}

```

Immutable; Jackson serializes it via component accessors. The static `of( ... )` stamps `now()`.

Custom exceptions — both extend `RuntimeException` (unchecked, so services throw freely and transactions roll back by default):

```

public class NotFoundException extends RuntimeException { ... } // → 404
public class ConflictException extends RuntimeException { ... } // → 409

```

GlobalExceptionHandler — `@RestControllerAdvice (= @ControllerAdvice + @ResponseBody)`, a global interceptor for all controllers. Each `@ExceptionHandler` maps an exception to a status, all funneling through one `build( ... )` helper that guarantees the identical `ApiError` shape:

Exception	Status	Message behavior
<code>AuthenticationException</code>	401	Passes message (failed login / bad-or-expired refresh). Mirrors the filter-level 401.
<code>AccessDeniedException</code>	403	Fixed: <i>"Access denied: your role does not permit this operation."</i> Catches <code>@PreAuthorize</code> denials during invocation.
<code>NotFoundException</code>	404	Passes message.
<code>ConflictException</code>	409	Passes message.
<code>DataIntegrityViolationException</code>	409	Fixed message — translates a raw DB constraint violation into a clean 409 (no SQL leakage).
<code>HttpMessageNotReadableException</code>	400	Fixed: <i>"Malformed or unreadable request body."</i> (bad JSON).
<code>MethodArgumentNotValidException</code>	400	Extracts the first field error → <code>field + " " + defaultMessage</code> . The <code>@Valid</code> failure path.
<code>IllegalArgumentException</code>	400	Passes message (e.g. malformed <code>location_code</code> , unsupported file/format).
<code>Exception</code> (catch-all)	500	Fixed: <i>"An unexpected error occurred."</i> — masks internals.

Design takeaway: business code throws semantic exceptions and relies on bean validation; this single advice centralizes mapping to HTTP status + uniform JSON, while masking internal details. The 401/403 handlers intentionally match the security-filter handlers so the error shape is identical whether the failure happens in the filter chain or during method invocation.

7. Security & Authentication

ShelfLife uses **stateless JWT authentication** with **method-level RBAC**. There is no server-side session — every request carries a short-lived signed **access token** (HS256 JWT) in the `Authorization` header. A separate **opaque, hashed refresh token** is persisted server-side so the long-lived credential can be revoked and rotated (something a self-contained JWT cannot do). Two roles: `CENTRAL_ADMIN` (global) and `HUB_PICKER` (assigned-warehouse scope).

7.1 End-to-end flow

Login → token pair

1. Client `POST /auth/login` with `{usernameOrEmail, password}`.
2. `AuthService.login` looks the user up by email-or-name, verifies the password with `BCrypt`, checks the account is `ACTIVE`.
3. It mints an **access token** (HS256 JWT) carrying `sub`, `role`, `email`, `warehouseIds`, `iss`, plus a **refresh token** (256-bit random string, only its SHA-256 hash persisted).
4. Returns `TokenResponse {accessToken, refreshToken, tokenType:"Bearer", expiresIn}`.

Authenticated requests — `Authorization: Bearer <accessToken>`. `JwtAuthenticationFilter` verifies signature/expiry, reconstructs an `AuthenticatedUser` principal + a `ROLE_<role>` authority, stores it in the `SecurityContext`. No DB hit on the hot path.

Refresh (rotation) — `POST /auth/refresh`: hash the presented token, look it up, reject if unknown/revoked/expired; **revoke it** and issue a brand-new pair. A used refresh token can never be replayed.

Logout (revocation) — `POST /auth/logout` (needs a valid access token): revoke the stored token only if it belongs to the caller; idempotent. Login and logout emit audit events.

Why opaque hashed refresh tokens, not JWT refresh tokens? A JWT is self-validating and therefore *cannot be revoked* before expiry without server state. Making the refresh token an opaque random string whose SHA-256 hash is stored gives the server a revocation/rotation list (`refresh_tokens`): logout, rotation, and "delete user ⇒ kill all sessions" become simple row updates. Access tokens stay JWTs so the per-request hot path needs no DB lookup. **Why stateless?** Horizontal scalability (no sticky sessions / shared session store) and a clean REST contract.

7.2 SecurityConfig — filter chain, session policy, RBAC

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@EnableConfigurationProperties(JwtProperties.class)
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s → s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth → auth
                .requestMatchers("/auth/login", "/auth/refresh").permitAll()
                .requestMatchers("/v3/api-docs", "/v3/api-docs/**", "/swagger-ui.html",
                    "/swagger-ui/**", "/swagger-resources/**", "/webjars/**").permitAll()
                .anyRequest().authenticated())
            .exceptionHandling(ex → ex
                .authenticationEntryPoint(authenticationEntryPoint)
                .accessDeniedHandler(accessDeniedHandler))
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
```

- `@EnableWebSecurity` turns on the web filter chain; `@EnableMethodSecurity` enables `@PreAuthorize` (what makes "every controller method carries an explicit `@PreAuthorize`" enforceable).
- `@EnableConfigurationProperties(JwtProperties.class)` binds `security.jwt.*`.
- `.csrf(disable)` — CSRF protection is off because the API is **token-based and stateless** (no cookies/session to forge). CSRF defenses target cookie-authenticated browser sessions.
- `STATELESS` — Spring never creates/uses an `HttpSession`; `SecurityContext` is rebuilt from the JWT each request.
- Whitelist: `/auth/login`, `/auth/refresh`, and the springdoc/Swagger paths → `permitAll()`. `/auth/logout` is **not whitelisted** (needs a token). `.anyRequest().authenticated()` for the rest; fine-grained role checks are layered per-method via `@PreAuthorize`.
- `.exceptionHandling` wires the custom 401 entry point and 403 handler.
- `.addFilterBefore(jwtAuthenticationFilter, ...)` puts the JWT filter ahead of the form-login slot so the context is populated before authorization runs.
- `passwordEncoder()` — `BCryptPasswordEncoder` (adaptive salted hashing).

7.3 JwtService — signing, claims, verification

```
@Service
public class JwtService {
    private final SecretKey signingKey;    // Keys.hmacShaKeyFor(Base64.decode(secret))

    public String generateAccessToken(User user, List<Long> warehouseIds) {
        Instant now = Instant.now();
        return Jwts.builder()
            .issuer(issuer)
            .subject(String.valueOf(user.getId()))
            .claims(Map.of("role", user.getRole().getName(),
                           "email", user.getEmail(),
                           "warehouseIds", warehouseIds))
            .issuedAt(Date.from(now))
            .expiration(Date.from(now.plus(accessTokenExpirationMinutes, ChronoUnit.MINUTES)))
            .signWith(signingKey)
            .compact();
    }

    public Claims parse(String token) {
        return Jwts.parser().verifyWith(signingKey).build()
            .parseSignedClaims(token).getPayload();
    }
}
```

- Uses **JWT**. The configured `secret` is Base64-decoded into raw bytes, then `Keys.hmacShaKeyFor( ... )` builds an HMAC `SecretKey`; JWT auto-selects **HS256** for a 256-bit key.
- **Claims:** `iss`, `sub` (user id as string), `role`, `email`, `warehouseIds` (the picker's mapped ids, or **empty** for **CENTRAL_ADMIN** = global scope — the data-isolation claim), `iat`, `exp`.
- **Verification:** `verifyWith` validates the HMAC signature; `parseSignedClaims` also enforces expiry. Any failure throws a `JwtException` caught by the filter.

7.4 JwtAuthenticationFilter — Bearer header → authenticated context

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain chain) throws ... {
        String header = request.getHeader("Authorization");
        if (header == null || !header.startsWith("Bearer ")) { chain.doFilter(request, response); return; }
        String token = header.substring("Bearer ".length()).trim();
        try {
            Claims claims = jwtService.parse(token);
            AuthenticatedUser principal = new AuthenticatedUser(
                Long.valueOf(claims.getSubject()),
                claims.get("email", String.class),
                claims.get("role", String.class),
                extractWarehouseIds(claims));
            var authority = new SimpleGrantedAuthority("ROLE_" + principal.role());
            var authentication = new UsernamePasswordAuthenticationToken(principal, null, List.of(authority));
            authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(authentication);
        } catch (JwtException | IllegalArgumentException ex) {
            SecurityContextHolder.clearContext();
        }
        chain.doFilter(request, response);
    }
}
```


- extends `OncePerRequestFilter` — runs exactly once per request.
- Missing/non-Bearer header → the filter does nothing and passes the request on (request stays anonymous; the entry point 401s it later if the endpoint is protected).
- Builds the `AuthenticatedUser` principal from claims, coerces the `warehouseIds` JSON array (`Number` → `Long`) defensively.
- Adds the **ROLE_ prefix** here — exactly what `hasRole('CENTRAL_ADMIN')` expects. Credentials are `null` (already verified via signature); supplying authorities makes the token *authenticated*.
- On **any** parse failure (bad/expired/malformed token), it **clears the context** and continues — a bad token is treated like no token (401 downstream, not 500).

7.5 AuthenticatedUser & JwtProperties

```
public record AuthenticatedUser(Long id, String email, String role, List<Long> warehouseIds) {
    public boolean isCentralAdmin() { return "CENTRAL_ADMIN".equals(role); }
}

@ConfigurationProperties(prefix = "security.jwt")
public record JwtProperties(String secret, long accessTokenExpirationMinutes,
                           long refreshTokenExpirationDays, String issuer) {}
```

`AuthenticatedUser` is the immutable `SecurityContext` principal — controllers inject it with `@AuthenticationPrincipal`, and `WarehouseScope` /SpEL reach it. `JwtProperties` binds the `security.jwt.*` config.

7.6 RefreshToken entity + repository

```
@Entity @Table(name = "refresh_tokens")
public class RefreshToken {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    @Column(name = "user_id", nullable = false) private Long userId;
    @Column(name = "token_hash", nullable = false, unique = true, length = 64) private String tokenHash;
    @Column(name = "expires_at", nullable = false) private OffsetDateTime expiresAt;
    @Column(name = "revoked", nullable = false) private boolean revoked = false;
    @Column(name = "created_at", insertable = false, updatable = false) private OffsetDateTime createdAt;
}
```

- `token_hash` stores the **SHA-256 hex** (length 64) of the opaque token — never the token itself. `unique=true` prevents duplicates.
- `revoked` is the rotation/logout kill-switch. `created_at` is DB-owned (`insertable=false`, `updatable=false`).
- Repo: `findByTokenHash(String)` (always look up by hash) and `deleteByUserId(Long)` (kill all of a user's sessions).

7.7 AuthController — endpoints

Method	Path	@PreAuthorize	Request	Response	Status
POST	/auth/login	<code>permitAll()</code>	<code>LoginRequest (@Valid)</code>	<code>TokenResponse</code>	200
POST	/auth/refresh	<code>permitAll()</code>	<code>RefreshRequest (@Valid)</code>	<code>TokenResponse</code>	200
POST	/auth/logout	<code>isAuthenticated()</code>	<code>LogoutRequest (@Valid) + @AuthenticationPrincipal</code>	<code>Void</code>	204

`permitAll()` on login/refresh matches the whitelist (no token yet). `isAuthenticated()` on logout requires a token, and the principal scopes the logout to the caller's id. Every method has an explicit `@PreAuthorize`.

7.8 AuthService — logic

- **login** (@Transactional): findByEmailOrName (accepts email or username); missing user or bad BCrypt match → BadCredentialsException("Invalid credentials.") (generic — no user enumeration); non- ACTIVE status → DisabledException ; mint access token (with warehouseIdsFor(user) — mapped ids for pickers, empty for admin) + refresh token; publish LOGIN audit; return TokenResponse.bearer(...) .
- **refresh** (@Transactional): look up by sha256Hex(token) ; reject if revoked/expired; **revoke the presented token** (rotation); issue a fresh pair.
- **logout** (@Transactional): look up by hash, .filter(t → t.getUserId().equals(currentUserId)) (only revoke your own token), .ifPresent(... revoke) ; idempotent; publish LOGOUT audit.
- **issueRefreshToken** : 32 random bytes from SecureRandom , Base64URL-encoded → the raw token returned to the client; persist only sha256Hex(raw) , expiresAt , revoked=false .

Why SHA-256 (not BCrypt) for the refresh token? The token is already a high-entropy 256-bit random value — nothing to brute-force — so a fast digest is sufficient and avoids BCrypt's deliberate slowness on every refresh. Passwords (low-entropy) use BCrypt.

7.9 401 vs 403 handlers

Both serialize the shared ApiError JSON, wired in SecurityConfig.exceptionHandling :

- **RestAuthenticationEntryPoint** (401) — no/invalid/expired token on a protected endpoint. Means "*who are you?*" (authenticate).
- **RestAccessDeniedHandler** (403) — authenticated but wrong role. Means "*I know who you are, you may not.*"

7.10 DTO validation

DTO	Field(s)	Annotation
LoginRequest	usernameOrEmail , password	@NotBlank
RefreshRequest	refreshToken	@NotBlank
LogoutRequest	refreshToken	@NotBlank

@NotBlank = non-null **and** at least one non-whitespace char; a violation aborts the request before the service runs → **400** via the global handler. TokenResponse is a response DTO (no validation) with factory TokenResponse.bearer(access, refresh, expiresIn) hardcoding tokenType = "Bearer" .

8. Users, Warehouses & Data Isolation

This module covers the RBAC user model, warehouse master data, and the **BRD 3.1 data-isolation** core (WarehouseScope).

8.1 Entities

User — note @ManyToOne to Role and the @Enumerated(STRING) status:

```

@Entity @Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    @Column(name = "name", nullable = false, length = 150) private String name;
    @Column(name = "email", nullable = false, unique = true, length = 255) private String email;
    @Column(name = "password_hash", nullable = false, length = 255) private String passwordHash;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    @JoinColumn(name = "role_id", nullable = false)
    private Role role;

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false, length = 20)
    private UserStatus status = UserStatus.ACTIVE;

    @Column(name = "created_at", nullable = false, insertable = false, updatable = false)
    private OffsetDateTime createdAt;
}

```

- `@Table(name = "users")` is needed because `user` is a reserved word in some DBs.
- `@GeneratedValue(ENTITY)` defers id generation to Postgres's identity column.
- **role relationship:** `@ManyToOne + @JoinColumn("role_id")`. `EAGER` because the service and `UserResponse.from(...)` dereference `user.getRole().getName()` outside a session — lazy would risk `LazyInitializationException`.
- **status:** `@Enumerated(EnumType.STRING)` stores the enum *name* ("ACTIVE"), not its ordinal — robust against reordering and matching the DB `CHECK`. Java-side default `ACTIVE`.
- **createdAt:** `insertable=false, updatable=false` — DB owns it (`now()`); JPA reads but never writes. `OffsetDateTime` maps `TIMESTAMPZ`.

`UserStatus { ACTIVE, INACTIVE, SUSPENDED }` and `Role (id + unique name, seeded CENTRAL_ADMIN / HUB_PICKER)` are plain. `Warehouse` has `id`, unique `warehouseCode` (e.g. `WH-MAIN`), `warehouseName`, nullable `address`.

PickerWarehouseMapping — the picker↔warehouse join table uses a **composite key** via `@EmbeddedId`:

```

@Entity @Table(name = "picker_warehouse_mapping")
public class PickerWarehouseMapping {
    @EmbeddedId private PickerWarehouseId id;
}

@Embeddable
public class PickerWarehouseId implements Serializable {
    @Column(name = "picker_id", nullable = false) private Long pickerId;
    @Column(name = "warehouse_id", nullable = false) private Long warehouseId;
    // hand-written equals()/hashCode() over both fields (mandatory for a JPA id class)
}

```

- **@EmbeddedId vs @IdClass**: here the entity holds one field of an `@Embeddable` type that is the key. (The `@IdClass` alternative would duplicate the two fields as separate `@Id` fields on the entity.)
- `implements Serializable + hand-written equals / hashCode` are **mandatory** for any composite-key class so the persistence context can compare/cache identity. The mapping entity has no other columns — a pure association table.

8.2 WarehouseScope — the BRD 3.1 data-isolation core

This `@Component` is the single enforcement point ensuring a `HUB_PICKER` can only touch warehouses in their JWT claims, and that **a client-supplied warehouse id can never widen access**. `CENTRAL_ADMIN` is global and exempt. Scope is read from the `AuthenticatedUser` principal — never from request parameters.

```
@Component
public class WarehouseScope {

    public Optional<List<Long>> resolveQueryScope(AuthenticatedUser principal, Long requestedWarehouseId) {
        if (principal.isCentralAdmin()) {
            return requestedWarehouseId == null
                ? Optional.empty() // no restriction (global read)
                : Optional.of(List.of(requestedWarehouseId)); // admin may filter to any one
        }
        List<Long> assigned = principal.warehouseIds();
        if (requestedWarehouseId == null) {
            return Optional.of(List.copyOf(assigned)); // default to picker's own scope
        }
        if (!assigned.contains(requestedWarehouseId)) {
            throw new AccessDeniedException("Warehouse " + requestedWarehouseId + " is outside your assigned scope.");
        }
        return Optional.of(List.of(requestedWarehouseId)); // narrow to one of *their* warehouses
    }

    public void assertAccessible(AuthenticatedUser principal, Long warehouseId) {
        if (principal.isCentralAdmin()) return;
        if (warehouseId == null || !principal.warehouseIds().contains(warehouseId)) {
            throw new AccessDeniedException("Warehouse " + warehouseId + " is outside your assigned scope.");
        }
    }
}
```

- `resolveQueryScope` (used for **read queries**) returns `Optional.empty()` = "no restriction" (only for an admin with no filter); a present list = the concrete set the query must be limited to. A picker with no filter is silently scoped to *all their* warehouses; a picker requesting a foreign id is **rejected (403)**, never honored; a picker requesting one of their own ids narrows to it. The crucial property: the only client-influenced value is treated as a *filter request* validated against the claims — it can narrow but **never widen**.
- `assertAccessible` (used for **single-warehouse mutations**) is the write-side analogue: admin passes; a picker with a `null` or non-assigned id throws 403.

8.3 Endpoints

Every method carries an explicit `@PreAuthorize`. No `/api` prefix.

`UserController` — `/users` (admin-only except `/me`):

Method & Path	@PreAuthorize	Request	Response	Notes
GET /users	hasRole('CENTRAL_ADMIN')	—	List<UserResponse>	
GET /users/{id}	hasRole('CENTRAL_ADMIN')	path id	UserResponse	404 if missing
POST /users	hasRole('CENTRAL_ADMIN')	@Valid CreateUserRequest	UserResponse (201)	
PUT /users/{id}	hasRole('CENTRAL_ADMIN')	@Valid UpdateUserRequest	UserResponse	
DELETE /users/{id}	hasRole('CENTRAL_ADMIN')	path id + principal	Void (204)	passes caller id to forbid self-deletion
GET /users/me	isAuthenticated()	principal	Map of id/email/role/warehouseIds	any authenticated user; reads the principal, no DB hit

WarehouseController — /warehouses (admin-only): GET (list) and POST (@Valid CreateWarehouseRequest → 201; 409 on duplicate warehouseCode). No update/delete.

PickerMappingController — POST /map-picker (admin-only): @Valid MapPickerRequest → PickerMappingResponse (the picker's full assigned set after the idempotent assignment).

8.4 Services (key methods)

UserService.create — pre-checks existsByEmail → 409, **BCrypt-hashes the password** (passwordEncoder.encode) before storing, resolves the role by name, defaults status to ACTIVE . **update** — uniqueness check via existsByEmailAndIdNot (excludes own row); password is **optional on update** (only re-hashed if a non-blank value is supplied). **delete** — self-deletion guard (id.equals(currentUserId) → IllegalArgumentException), then removes dependent rows (mappings, refresh tokens) before deleting the user (FK safety).

PickerWarehouseService.assign — loads the user or 404s; **role gate** (only HUB_PICKER can be mapped, else IllegalArgumentException); for each warehouse id validates existence (404) and inserts the PickerWarehouseId **only if it doesn't already exist** → **idempotent**; returns the picker's full set via findWarehouseIdsByPickerId (also what populates the JWT warehouseIds claim at login).

8.5 Repositories

- UserRepository: findByEmail , findByEmailOrName (login identifier may be email or name), existsByEmail , existsByEmailAndIdNot .
- RoleRepository: findByName .
- WarehouseRepository: existsByWarehouseCode , findByWarehouseCode .
- PickerWarehouseMappingRepository extends JpaRepository<PickerWarehouseMapping, PickerWarehouseId> (note the composite id type) with JPQL that navigates the embedded id:

```
@Query("select m.id.warehouseId from PickerWarehouseMapping m where m.id.pickerId = :pickerId")
List<Long> findWarehouseIdsByPickerId(@Param("pickerId") Long pickerId);

@Modifying @Query("delete from PickerWarehouseMapping m where m.id.pickerId = :pickerId")
void deleteByPickerId(@Param("pickerId") Long pickerId);
```

8.6 DTO validation

- `CreateUserRequest`: `name` `@NotBlank @Size(max=150)`; `email` `@NotBlank @Email`; `password` `@NotBlank @Size(min=8)`; `role` `@NotBlank`; `status` *optional* (service defaults to ACTIVE).
- `UpdateUserRequest`: same but `status` is `@NotNull` (required), and `password` carries **only** `@Size(min=8)` (optional — null leaves it unchanged).
- `CreateWarehouseRequest`: `warehouseCode` `@NotBlank @Size(max=50)`, `warehouseName` `@NotBlank @Size(max=150)`, `address` *optional* `@Size(max=500)`.
- `MapPickerRequest`: `pickerId` `@NotNull`, `warehouseIds` `@NotEmpty`.
- Responses (`UserResponse`, `WarehouseResponse`, `PickerMappingResponse`) have no validation and **never expose** `passwordHash`.

9. Inventory: Products & Shelf Locations

This feature owns two pieces of master data: **products** (SKUs) and **shelf locations** (where each SKU physically lives in a warehouse). It implements BRD 3.2 (Master Location Mapping) on writes and participates in BRD 3.1 (Data Isolation) on reads.

9.1 Entities

Product — a **natural String primary key**, no `@GeneratedValue`:

```
@Entity @Table(name = "products")
public class Product {
    @Id @Column(name = "sku", length = 64) private String sku;
    @Column(name = "name", nullable = false, length = 255) private String name;
    @Column(name = "barcode", length = 128) private String barcode;
    @Column(name = "unit", length = 32) private String unit;
}
```

Because the id is an *assigned* `String` (never null for a new row), Hibernate can't use "id is null ⇒ INSERT". This is exactly why `ProductService.create` checks existence *before* saving rather than relying on JPA to detect novelty.

ShelfLocation — generated `Long` PK, FKs as plain scalar columns, unique (`warehouse_id`, `sku`):

```

@Entity @Table(name = "shelf_locations")
public class ShelfLocation {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    @Column(name = "warehouse_id", nullable = false) private Long warehouseId;
    @Column(name = "sku", nullable = false, length = 64) private String sku;
    @Column(name = "aisle", length = 32) private String aisle;
    @Column(name = "bay", length = 32) private String bay;
    @Column(name = "shelf", length = 32) private String shelf;
    @Column(name = "location_code", nullable = false, length = 64) private String locationCode;
}

```

- FKs (`warehouseId` , `sku`) are modeled as **raw columns, not @ManyToOne** — relational integrity is enforced in the DB and the **service validates references explicitly** (keeps the entity light, avoids accidental lazy loading).
- The unique (`warehouse_id` , `sku`) is the heart of "Master Location Mapping": one shelf per product per warehouse. The service pre-empts the constraint with a friendly 409.
- `aisle / bay / shelf` are **derived** from `location_code` (not accepted from the client).

9.2 LocationCodes + ParsedLocation — regex validation

```

public final class LocationCodes {
    public static final String FORMAT_EXAMPLE = "Aisle_A-Bay_04-Shelf_2";
    private static final Pattern PATTERN =
        Pattern.compile("^Aisle_([A-Za-z0-9]+)-Bay_([0-9]+)-Shelf_([0-9]+)$");

    public static ParsedLocation parse(String locationCode) {
        if (locationCode == null || locationCode.isBlank())
            throw new IllegalArgumentException("location_code must not be blank; expected '" + FORMAT_EXAMP
Matcher m = PATTERN.matcher(locationCode.trim());
        if (!m.matches())
            throw new IllegalArgumentException("Invalid location_code '" + locationCode + "': expected ... "
        return new ParsedLocation(m.group(1), m.group(2), m.group(3));
    }
}

```

The regex `^Aisle_([A-Za-z0-9]+)-Bay_([0-9]+)-Shelf_([0-9]+)$` dissected:

- `^ ... $` anchor the *whole* string (no junk).
- `Aisle_` literal, then group 1 = **aisle** `[A-Za-z0-9]+` (alphanumeric).
- `-Bay_` literal, then group 2 = **bay** `[0-9]+` (digits only).
- `-Shelf_` literal, then group 3 = **shelf** `[0-9]+` (digits only).
- Compiled once into a `static final Pattern` (thread-safe, compile-once). `parse` trims first, throws `IllegalArgumentException` (→ 400) on blank/no-match, else returns the three captured groups as a `ParsedLocation(aisle, bay, shelf)` record.

Accuracy note (a common misconception): there is **no zero-padding code** in this package. The regex accepts numeric bay/shelf of *any* length and performs no padding. Seed values like `Bay_04` are padded *by the data author*, which is what makes the stored `location_code` strings sort correctly lexicographically (the picker route sorts on that string). The code neither adds nor enforces the padding.

9.3 Services

ProductService — `listAll()` (`readOnly`) maps to `ProductResponse`; **create** checks `existsById(sku)` → `ConflictException` (409) before saving (necessary because of the natural assigned id).

ShelfLocationService (collaborators: `shelf repo`, `WarehouseRepository`, `ProductRepository`, `WarehouseScope`):

- **list(principal, requestedWarehouseId)** (`readOnly`) — data isolation via `WarehouseScope.resolveQueryScope: Optional.empty()` ⇒ admin unrestricted (`findAll`); a scoped list ⇒ `findByWarehouseIdInOrderById(scopedIds)` (or empty list if the scope is empty). A picker requesting a foreign warehouse is rejected 403 inside `resolveQueryScope` — the filter is always derived from JWT claims, never trusted from the param.
- **create(request)** — `LocationCodes.parse` first (400 on malformed), `validateReferences` (404 on unknown warehouse/sku), then `existsByWarehouseIdAndSku` → **409** if a mapping already exists (message points to PUT). Builds the entity, applies parsed components + trimmed code, saves.
- **update(request)** — same parse + reference validation, then loads the existing mapping by (`warehouseId`, `sku`) or **404** (the inverse of create's 409), re-applies and saves.
- Helpers: `validateReferences` enforces FK existence in app code (since no `@ManyToOne`); `applyLocation` copies aisle/bay/shelf + trimmed code (shared by create/update).

9.4 Endpoints — the deliberate singular/plural split

Method	Path	@PreAuthorize	Notes
GET	/products	<code>hasRole('CENTRAL_ADMIN')</code>	list products
POST	/products	<code>hasRole('CENTRAL_ADMIN')</code>	@Valid CreateProductRequest → 201 ; 409 if SKU exists
GET	/shelf-locations (plural)	<code>hasAnyRole('CENTRAL_ADMIN', 'HUB_PICKER')</code>	picker-facing read; optional ? <code>warehouseId=</code> ; service enforces isolation → 403 out-of-scope. The only inventory endpoint a picker may call.
POST	/shelf-location (singular)	<code>hasRole('CENTRAL_ADMIN')</code>	@Valid ShelfLocationRequest → 201 ; 400/404/409
PUT	/shelf-location (singular)	<code>hasRole('CENTRAL_ADMIN')</code>	update; 404 if no mapping exists

The plural read vs singular write naming is intentional and documented in the controller Javadoc.

9.5 Repositories & DTOs

- `ProductRepository` extends `JpaRepository<Product, String>` (id type is `String`).
- `ShelfLocationRepository`: `findByWarehouseIdInOrderById(Collection<Long>)`, `findByWarehouseIdAndSku`, `existsByWarehouseIdAndSku` (the 409 check), `findByWarehouseIdAndSkuIn` (batch, used by routing).
- `CreateProductRequest`: `sku @NotBlank @Size(max=64)`, `name @NotBlank @Size(max=255)`, `barcode / unit optional @Size`.

- `ShelfLocationRequest`: `warehouseId @NotNull`, `sku @NotBlank`, `locationCode @NotBlank` (format validated in the service, not by Bean Validation). **aisle / bay / shelf are absent** — derived server-side so the client can't send inconsistent components.

Two ID strategies side by side (interview point): `Product` uses a natural assigned `String` PK (forcing explicit `existsById` before save); everything else uses a generated `Long` IDENTITY PK.

10. Order Ingestion

The `orders` feature (BRD 3.2) lets a Central Admin bulk-upload a CSV or Excel file of order lines. The file is parsed by a pluggable parser, every row is validated independently, rows are grouped by `Order_ID`, and each group is persisted **atomically** (all-or-nothing per order). The client always gets a structured per-row report.

10.1 Entities — `Order` and `OrderItem`

```
@Entity @Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    @Column(name = "order_number", nullable = false, unique = true, length = 64) private String orderNumber;
    @Column(name = "customer_id", length = 64) private String customerId;
    @Column(name = "warehouse_id", nullable = false) private Long warehouseId;
    @Column(name = "picker_id") private Long pickerId;          // set when claimed; null while PENDING

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false, length = 20)
    private OrderStatus status = OrderStatus.PENDING;

    @Version @Column(name = "version", nullable = false) private Integer version;
    @Column(name = "created_at", insertable = false, updatable = false) private OffsetDateTime createdAt;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>();

    public void addItem(OrderItem item) { item.setOrder(this); items.add(item); }
}
```

- `orderNumber` holds the *external* `Order_ID`; `unique=true` enforces no duplicate orders.
- **@Version** is the JPA **optimistic-lock** column. On a dirty update Hibernate appends `WHERE version = ?` and increments it; a stale version throws `OptimisticLockException`. This is what makes the concurrent picker **claim** safe (see §11.3).
- **@OneToMany(mappedBy="order", cascade=ALL, orphanRemoval=true)** — the *inverse* side; `OrderItem.order` owns the FK. Cascade lets the service build a full `Order` + items in memory and `save(order)` **once** to persist the whole aggregate. `addItem( ... )` syncs both sides of the bidirectional link.

`OrderItem` is the **owning** side: `@ManyToOne(fetch = LAZY, optional = false) + @JoinColumn("order_id")`, with `pickedQuantity` default 0 and `status` default `PENDING`. Enums `OrderStatus` {`PENDING`, `ASSIGNED`, `PICKING`, `COMPLETED`, `DISPATCHED`} and `OrderItemStatus` {`PENDING`, `PICKED`, `SKIPPED`} mirror the DB CHECK constraints.

10.2 File parsing — Strategy + Factory pattern

Parsing is decoupled via the **Strategy pattern** (`OrderFileParser` interface, multiple impls) + a **Factory** (`OrderFileParserFactory`) that picks by extension. Adding a new format = add one `@Component` implementing the interface, **no change to service or factory**.

```
public interface OrderFileParser {
    boolean supports(String filename);           // by extension
    List<RawOrderRow> parse(byte[] content);     // header excluded, blank rows skipped
}
```

Implementations throw `IllegalArgumentException` for **file-level** problems (bad header, empty file, unreadable) — that exception type is what distinguishes a **400** (whole upload rejected) from a per-row failure (200 + report).

- **CsvOrderFileParser** (OpenCSV) — `supports` keys off `.csv`; reads UTF-8, validates the header, trims cells, skips blank rows.
- **ExcelOrderFileParser** (Apache POI) — `supports` `.xlsx` / `.xls`; `WorkbookFactory.create` auto-detects HSSF/XSSF; reads the **first sheet only**. The key detail is **numeric-cell normalization**: Excel stores `5` as `5.0`, so `numericToString` converts whole numbers back to clean integer strings (`5.0` → `"5"`) — without this, the service's `Integer.parseInt("5.0")` would reject every quantity.
- **OrderFileParserFactory** — Spring injects **all** `OrderFileParser` beans into a `List`; `forFile(filename)` returns the first whose `supports( ... )` is true, else throws (400). Textbook Factory-over-Strategy.

Header validation lives in `OrderUploadColumns`: the six required columns (`Order_ID`, `Customer_ID`, `Warehouse_ID`, `Item_SKU`, `Item_Name`, `Quantity_Ordered`) must match **exactly** (case-insensitive, trimmed; missing or unexpected or duplicate columns all throw). `indexHeader` returns a `Map<canonicalColumn → index>` so parsers don't depend on column order.

10.3 Endpoint — POST /orders/upload

```
@PostMapping(path = "/upload", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
@PreAuthorize("hasRole('CENTRAL_ADMIN')")
public OrderUploadReport upload(@RequestParam("file") MultipartFile file) throws IOException {
    if (file == null || file.isEmpty()) throw new IllegalArgumentException("No file uploaded ...");
    return orderIngestionService.ingest(file.getOriginalFilename(), file.getBytes());
}
```

The central design split — 200 vs 400:

- **File-level errors** → **400**. Empty file, unsupported extension, bad/missing/duplicate/unexpected header, unreadable content (all `IllegalArgumentException` → 400). No report.
- **Row-level failures** → **200 + report**. Once parsing succeeds, the batch never fails wholesale; each row gets an outcome in the `OrderUploadReport`, returned 200 even if rows FAILED/SKIPPED.

10.4 OrderIngestionService — the atomic grouping rule

`ingest( ... )` (`@Transactional`): pick parser → parse → validate every row into a `RowState` (with three **caches** memoizing warehouse/shelf/order-exists lookups so a 1000-row file doesn't hammer the DB) → group by `Order_ID` (blank `Order_ID` → FAILED immediately) → `processGroup` each → sort results by file row → assemble report.

`validateRow` — first failure wins: required fields non-blank; `Quantity_Ordered` parses as a positive int; `Order_ID` not already used (`existsByOrderNumber`); `Warehouse_ID` resolved as a warehouse *code* via `findByWarehouseCode` (the file never carries the DB id); the SKU must already have a **shelf location in that warehouse** (`existsByWarehouseIdAndSku`) — you can't order an item with nowhere to pick it from.

`processGroup` — the heart of **atomic-per-order**:

- If **any** row in the group failed → order **not** created; each bad row → FAILED, each valid sibling → **SKIPPED** (naming the offending line numbers). Nothing persisted.
- If all valid but the group has **>1 distinct Customer_ID or Warehouse_ID** → whole group FAILED ("Inconsistent ..."). Nothing persisted.
- Only if all valid *and* consistent → `persistOrder` (one `save(order)` cascades to items), every row SUCCESS, `ordersCreated++` .

This guarantees an order is all-or-nothing: you never get a half-ingested order.

10.5 DTOs

- `RowResultStatus { SUCCESS, FAILED, SKIPPED }` — SKIPPED means *this row was valid but its order was rejected because a sibling failed*.
- `OrderUploadRowResult(row, orderId, status, error)` with `success/failed/skipped` factories.
- `OrderUploadReport(totalRows, successCount, failedCount, skippedCount, ordersCreated, rows)` — counts tallied in its factory so they always reconcile with `rows` .

10.6 OrderRepository (also used by picking)

```
boolean existsByOrderNumber(String orderNumber);           // ingestion duplicate check
List<Order> findByWarehouseIdAndStatusOrderByCreatedAtAsc( ... ); // dashboard available queue
// ... picker dashboard derived queries ...

@Modifying(clearAutomatically = true, flushAutomatically = true)
@Query("update Order o set o.status = ... ASSIGNED, o.pickerId = :pickerId, o.version = o.version + 1 "
      + "where o.id = :orderId and o.status = ... PENDING and o.version = :expectedVersion")
int claim(@Param("orderId") Long orderId, @Param("pickerId") Long pickerId,
         @Param("expectedVersion") int expectedVersion);
```

The `claim` query (covered in §11.3) is the optimistic-lock conditional UPDATE; it returns the affected-row count (`1` = won, `0` = lost the race → 409).

11. Picker Workflow & Scan-to-Pick

This covers BRD 3.3 (Hub Picker Workflow) and the BRD 3.2 routing it depends on: warehouse entry, the dashboard, claiming under an optimistic lock, the optimized route, and the Scan-to-Pick loop with skip/idempotency.

11.1 Domain model

`PickerActiveWarehouse` — server-side "which warehouse am I working in" state. Because auth is **stateless JWT** (no HTTP session), the active-warehouse choice is persisted one row per picker (`@Id` on `pickerId` , *no* `@GeneratedValue`). Re-selecting overwrites the row.

PickSession — a picker's working session for **one order**; opened on the first scan and closed (`COMPLETED` , `endTime` set) when the order is fully picked. `@Enumerated(String)` status from `PickSessionStatus` {`OPEN`, `IN_PROGRESS`, `COMPLETED`, `CANCELLED`} .

PickLog — an **immutable** record of one scan. `scanTime` is DB-owned (`insertable=false` , `updatable=false`); `scanId` is `unique=true` — the DB-level constraint backing scan idempotency.

Accurate caveat: `PickSession.status` defaults to `OPEN` , but `openOrGetSession` creates/looks up sessions as `IN_PROGRESS` and completes them with `COMPLETED` , so `OPEN` is never actually persisted by this flow.

11.2 Warehouse entry & dashboard

- **ActiveWarehouseService.select** — `warehouseScope.assertAccessible(picker, warehouseId)` (403 if not mapped) → load warehouse (404) → upsert the `PickerActiveWarehouse` row. The active warehouse **must** be one of the picker's mapped warehouses (BRD 3.1 isolation, from JWT claims).
- **requireActiveWarehouseId** — resolves the active warehouse for scoped reads; throws `ConflictException` (409) if none selected, and **re-validates** via `assertAccessible` (guards a mapping revoked after selection).
- **PickerDashboardService.dashboard** (`readOnly`) — entirely scoped to the active warehouse: **available** = unclaimed `PENDING` orders (shared queue, oldest first), **current** = this picker's `ASSIGNED` / `PICKING` , **completed** = this picker's `COMPLETED` (newest first).

Endpoints (all `hasRole('HUB_PICKER')`): `GET /picker/warehouses` , `POST /picker/select-warehouse` (`@Valid SelectWarehouseRequest`), `GET /picker/dashboard` .

11.3 `POST /orders/{id}/claim` — the optimistic lock

Many pickers see the same `PENDING` order; only one may claim it. The lock is a **conditional UPDATE**, not entity-`@Version` flush:

```
@Modifying(clearAutomatically = true, flushAutomatically = true)
@Query("update Order o set o.status = ... ASSIGNED, o.pickerId = :pickerId, o.version = o.version + 1 "
      + "where o.id = :orderId and o.status = ... PENDING and o.version = :expectedVersion")
int claim(Long orderId, Long pickerId, int expectedVersion);
```

- `@Modifying` → executes `executeUpdate()` and returns the **affected-row count**.
- `flushAutomatically=true` flushes pending changes *before* the UPDATE; `clearAutomatically=true` clears the persistence context *after* (the bulk UPDATE bypasses the first-level cache, so a loaded `Order` must be detached to avoid staleness).
- The `WHERE` embeds **two conditions** — `status = PENDING` *and* `version = :expectedVersion`; the `SET` flips status, stamps `pickerId` , and **increments version** .

`OrderClaimService.claim` (`@Transactional`): load or 404 → `assertAccessible` (403, isolation) → fast-path 409 if not `PENDING` → run `claim(orderId, pickerId, order.getVersion())` → `updated == 0` ⇒ **lost the race** ⇒ **409** → reload (context was cleared) → return `ClaimResponse` .

Why this beats entity `@Version` flush:

- **Atomic at the DB, no read-modify-write window in Java** — the check-and-set is one SQL statement.
- **Encodes the business precondition** `status = PENDING` , not just version equality.

- **Deterministic** — a lost race is `updated == 0` (a value to branch on), not a thrown `OptimisticLockException` to catch/translate. Cleaner on a hot, contended path.

(Verified by a 2-thread `OrderClaimConcurrencyTest`: exactly 1 success + 1 conflict, final status `ASSIGNED` / version 1.)

11.4 GET /orders/{id}/route — the optimized path (BRD 3.2)

`RoutingService.route (readOnly)`: load order (404) → `assertAccessible` (403) → join each `OrderItem` to its `ShelfLocation` (per warehouse + SKU; unmapped items get `null` codes) → **sort by location_code ascending with Comparator.nullsLast** (unmapped items sort last) → stamp a 1-based `sequence`. Because codes are zero-padded (`Aisle_A-Bay_01` → `Aisle_A-Bay_02` → `Aisle_B-Bay_01` → `Aisle_C-Bay_02`), lexicographic string order == travel order. Each `RouteStep` carries `itemId` and `status` so the client can identify the current step and render `PICKED/SKIPPED` state. Route is readable by `HUB_PICKER` **and** `CENTRAL_ADMIN`.

11.5 Scan-to-Pick: POST /scan

`ScanService.scan (@Transactional)` is the performance-critical path. The transaction does only essentials; **audit is deferred to an async listener (never inline)**.

```
// (A) Idempotency: a previously-recorded scanId is a no-op replay (duplicate=true), no mutation.
if (scanId != null && pickLogRepository.findByScanId(scanId).isPresent()) { ... return duplicate ... }

Order order = loadOwnedOrder(request.orderId(), picker); // 404 / 403 / not-your-order
ensureActive(order); // 409 if not ASSIGNED/PICKING
PickSession session = openOrGetSession(order, picker); // open on first scan; ASSIGNED→PICKING

OrderItem step = currentStep(order, codes) // next PENDING by location_code; else first SKI
    .orElseThrow() → new ConflictException("No remaining items to pick ...");

// (B) Current-step enforcement: scanned SKU must equal the expected step's SKU.
if (!step.getSku().equals(request.sku()))
    throw new ConflictException("Scan rejected: expected SKU '" + step.getSku() + "' ... but scanned '" + r

// (C) Correct scan: increment by exactly 1, log it.
step.setPickedQuantity(step.getPickedQuantity() + 1);
recordPickLog(session.getId(), step.getSku(), location, scanId);
step.setStatus(step.getPickedQuantity() ≥ step.getOrderedQuantity()
    ? OrderItemStatus.PICKED : OrderItemStatus.PENDING);

if (allPicked(order)) { // every item PICKED → order COMPLETED
    order.setStatus(OrderStatus.COMPLETED);
    session.setStatus(PickSessionStatus.COMPLETED);
    session.setEndTime(OffsetDateTime.now());
}

// (D) Audit deferred – publish events; @Async @TransactionalEventListener writes audit_logs AFTER commit.
eventPublisher.publishEvent(AuditEvent.of(AuditAction.SCAN, picker.id(), order.getId(), step.getSku(), loca
if (itemPicked) eventPublisher.publishEvent(AuditEvent.of(AuditAction.ITEM_PICKED, ...));
if (orderCompleted) eventPublisher.publishEvent(AuditEvent.of(AuditAction.ORDER_COMPLETED, ...));
```

Key behaviors:

- **(A) Idempotency** — known `scanId` → no-op replay; the `pick_logs.scan_id` `UNIQUE` constraint is the durable backstop (a concurrent duplicate that slips past the read fails on insert and rolls back). Rationale: rugged scanners on flaky networks retry.

- **Ownership/state guards** — `loadOwnedOrder` 404s missing, 403s out-of-scope, 403s if `pickerId ≠ caller`; `ensureActive` 409s if `PENDING` ("claim it first") or already `COMPLETED` / `CANCELLED`.
- **currentStep** — the next `PENDING` item by `location_code`; if none, the first `SKIPPED` item (so skips are revisited and the picker is **never blocked**).
- **(B) Wrong SKU** → **409** naming the expected SKU/location.
- **(C)** increment by 1, one `PickLog`; item → `PICKED` when `picked ≥ ordered`, else back to `PENDING` (a revisited `SKIPPED` item becomes active again).
- **Completion** flips order + session to `COMPLETED`.

11.6 POST /orders/{id}/items/{itemId}/skip

`ScanService.skip` (`@Transactional`): same ownership/state guards → find the item in the order (404) → reject if already `PICKED` (409) → set `SKIPPED` → publish `SKIP` audit. `currentStep` then advances past it (returning to skipped items only after all `PENDING` are done).

11.7 Endpoint reference (picking)

Method	Path	@PreAuthorize	Notable statuses
GET	/picker/warehouses	<code>hasRole('HUB_PICKER')</code>	200
POST	/picker/select-warehouse	<code>hasRole('HUB_PICKER')</code>	200; 403 not mapped; 404 missing
GET	/picker/dashboard	<code>hasRole('HUB_PICKER')</code>	200; 409 no active warehouse
POST	/orders/{id}/claim	<code>hasRole('HUB_PICKER')</code>	200; 404; 403 out-of-scope; 409 already claimed / lost race
GET	/orders/{id}/route	<code>hasAnyRole('CENTRAL_ADMIN', 'HUB_PICKER')</code>	200; 404; 403
POST	/scan	<code>hasRole('HUB_PICKER')</code>	200; 404; 403; 409 not-claimed / wrong-SKU / nothing-left
POST	/orders/{id}/items/{itemId}/skip	<code>hasRole('HUB_PICKER')</code>	200; 404; 403; 409 not-pickable / already-picked

Scan/skip are `HUB_PICKER`-only with **ownership enforced in the service** (not in `@PreAuthorize`).

11.8 Concurrency & transaction boundaries

- Reads `@Transactional(readOnly = true)`; mutations `@Transactional` (whole method wraps a transaction; mutations flush atomically at commit).
- **Claim contention** resolved by the DB-atomic conditional `UPDATE` (§11.3): exactly one writer wins, the rest get `updated == 0` → 409.
- **Scan idempotency** is two-layered: early `findByScanId` short-circuit + `pick_logs.scan_id` `UNIQUE` backstop.

- **Hot-path minimalism:** the scan tx does validate → increment → one insert → (maybe) close session; audit is pushed off-thread (after commit), so audit latency/failures never block a scan.
- **Data isolation** on every order-scoped op via `WarehouseScope.assertAccessible` (claim, route, scan, skip), plus scan/skip require the order's `pickerId == caller`.

12. Audit Trail & Dispatch Reporting

12.1 Audit Trail (BRD §4) — event-driven, off the request thread

The audit trail is built so **no business path ever blocks on writing an audit row**:

```

service mutates data → eventPublisher.publishEvent(AuditEvent)  (inside the business tx)
                        |
tx COMMITS → @TransactionalEventListener fires (AFTER_COMMIT)
                        |
@Async("auditExecutor") hops to a dedicated thread pool
                        |
JpaAuditLogWriter.write() - REQUIRES_NEW tx - INSERT audit_logs

```

AuditEvent — an immutable record carrying the BRD §4 fields:

```

public record AuditEvent(AuditAction action, Long userId, Long orderId,
                        String sku, String location, OffsetDateTime occurredAt) {
    public static AuditEvent of(AuditAction action, Long userId) { ... now() ... }
    public static AuditEvent of(AuditAction action, Long userId, Long orderId, String sku, String location)
}

```

`occurredAt` is captured with `now()` **when the action happens** (inside the request), not when the async write later occurs — so the persisted timestamp reflects the *action*, not the write.

Publishers inject `ApplicationEventPublisher` and call `publishEvent(...): AuthService` (LOGIN/LOGOUT), `ScanService` (SCAN, ITEM_PICKED, ORDER_COMPLETED, SKIP), `ReportService` (DOWNLOAD_REPORT).

The listener — the heart of the design:

```

@Component
public class AuditEventListener {
    @Async("auditExecutor")
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT, fallbackExecution = true)
    public void onAuditEvent(AuditEvent event) {
        auditLogWriter.write(event);
    }
}

```

- `@Async("auditExecutor")` — runs on the named pool (from `AsyncConfig`), **off the caller's thread**.
- `@TransactionalEventListener(AFTER_COMMIT)` — handles the event **only after the publishing transaction commits**. If the business tx rolls back (e.g. an optimistic-lock conflict), the event is dropped and **no audit row is written** — the log only ever contains actions that actually took effect.
- `fallbackExecution = true` — still runs if the event is published outside any transaction (a safety net).

The writer — interface `seam ( AuditLogWriter )` + JPA impl `JpaAuditLogWriter`, annotated

`@Transactional(propagation = Propagation.REQUIRES_NEW)` — it opens a **brand-new transaction** for the insert (necessary because, firing `AFTER_COMMIT`, the original tx is already gone). `AuditLog` entity stores `action` as a `VARCHAR(100)` string (the enum name). `AuditAction { LOGIN, LOGOUT, SCAN, SKIP, ITEM_PICKED, ORDER_COMPLETED, DOWNLOAD_REPORT }`.

Design rationale & trade-offs:

- **Why off the request thread?** The scan path is latency-critical; an inline audit insert would add a DB round-trip to every scan.
- **Why AFTER_COMMIT?** Audit integrity — never log an action that was subsequently rolled back.
- **Durability trade-off (the honest caveat):** this is an **in-process async** pipeline. Between commit and the audit insert, the event lives only in the JVM's executor queue. If the JVM crashes (or the pool rejects under extreme load) in that window, **that audit event is lost**. For a hardened, exactly-once trail you'd use a **transactional outbox** (write an outbox row in the *same* tx, relay separately) or a durable **message broker** (Kafka/RabbitMQ). The current design trades strict durability for low request latency — appropriate at this stage, with a clear upgrade path.

12.2 Dispatch Report (BRD 3.4) — GET /report

```
@GetMapping
@PreAuthorize("hasRole('CENTRAL_ADMIN')")
public ResponseEntity<?> report(
    @RequestParam(required = false) @DateTimeFormat(iso = ISO.DATE) LocalDate date,
    @RequestParam(required = false) String warehouse,
    @RequestParam(required = false) Long picker,
    @RequestParam(defaultValue = "json") String format,
    @AuthenticationPrincipal AuthenticatedUser admin) {
    List<ReportRow> rows = reportService.generate(admin.id(), date, warehouse, picker);
    return switch (format.toLowerCase()) {
        case "json" → ResponseEntity.ok(rows);
        case "csv" → download(reportService.toCsv(rows), "dispatch-report.csv", "text/csv");
        case "xlsx" → download(reportService.toExcel(rows), "dispatch-report.xlsx", XLSX_CONTENT_TYPE);
        default → throw new IllegalArgumentException("Unsupported format ...");
    };
}
```

- Admin-only. Optional filters `date` (ISO date), `warehouse` (code), `picker` (id). **Output negotiation is driven by the format query param** (not the `Accept` header) — JSON / CSV (OpenCSV) / XLSX (POI). `admin.id()` is forwarded so the `DOWNLOAD_REPORT` audit records *who* pulled it. `download( ... )` sets `Content-Disposition: attachment`.

The dynamic JPQL query builds the WHERE clause with a `StringBuilder`, appending each filter **only when its parameter is non-null**, then binding only what it appended:


```

StringBuilder jpql = new StringBuilder("""
    select new com.ashu.shelflife.reports.dto.ReportRow(
        o.orderNumber, o.pickerId, w.warehouseCode,
        oi.sku, oi.itemName, oi.orderedQuantity, oi.pickedQuantity)
    from OrderItem oi join oi.order o, Warehouse w
    where w.id = o.warehouseId
    """);
if (startDate != null) jpql.append(" and o.createdAt ≥ :startDate and o.createdAt < :endDate");
if (warehouseCode != null) jpql.append(" and w.warehouseCode = :warehouse");
if (pickerId != null) jpql.append(" and o.pickerId = :picker");
jpql.append(" order by o.orderNumber, oi.sku");
// ... bind only the appended params ...

```

- **Why dynamic append rather than `:param is null or column = :param`?** On PostgreSQL the static null-guard fails — PG **can't infer the type of a NULL bind parameter** ("could not determine data type of parameter \$N"). Appending the clause only when the value is present binds the parameter only where its type is unambiguous (and lets PG plan a tighter query).
- **The joins** mix an **association join** (`join oi.order o`) and a **theta join** (`, Warehouse w ... where w.id = o.warehouseId`) — the theta join is needed because `Order` stores `warehouseId` as a plain column rather than a mapped `@ManyToOne`. It lets the report project `w.warehouseCode` (the human `Warehouse_ID`) instead of the raw FK.
- **Date filtering** uses a half-open range `[startOfDay, startOfNextDay)` (a single `LocalDate` expanded in the system zone) — robust against a `TIMESTAMPTZ` column.
- **Constructor projection** (`select new ...ReportRow( ... )`) yields typed DTOs and computes the fulfillment rate during construction.

ReportRow — BRD-exact JSON keys + Java-side rate:

```

public record ReportRow(
    @JsonProperty("Order_ID") String orderId,
    @JsonProperty("Picker_ID") Long pickerId,
    @JsonProperty("Warehouse_ID") String warehouseId,
    @JsonProperty("Item_SKU") String itemSku,
    @JsonProperty("Item_Name") String itemName,
    @JsonProperty("Quantity_Ordered") Integer quantityOrdered,
    @JsonProperty("Quantity_Picked") Integer quantityPicked,
    @JsonProperty("Fulfillment_Rate") double fulfillmentRate) {

    public static double fulfillmentRate(Integer picked, Integer ordered) {
        if (ordered == null || ordered ≤ 0) return 0.0; // defensive on zero/null
        return (double) (picked == null ? 0 : picked) / ordered;
    }
}

```

- `@JsonProperty( ... )` forces Jackson to emit the **exact BRD field names** (matching the CSV/Excel headers), not Java camelCase.
- **Fulfillment_Rate is computed in Java, not SQL** — the 7-arg projection constructor delegates to compute it, avoiding per-DB division/NULL semantics and keeping the divide-by-zero guard in one tested place.

Rendering (`ReportService`) — one shared `HEADERS` array drives all three formats. `generate( ... )` is `@Transactional(readOnly = true)`, expands `date` → `[start,end)`, runs the query, **publishes the DOWNLOAD_REPORT audit event**, returns rows. `toCsv` (OpenCSV) formats the rate to 4 decimals with a fixed `Locale.US`; `toExcel` (POI `XSSFWorkbook`) writes numeric cells as real numbers (full `double` precision). Both wrap `IOException` as `UncheckedIOException`.

13. The Frontend

A **single** Vite + React 18 + TypeScript + Material UI v5 SPA in `frontend/`, serving both the admin console (`/admin/*`) and the picker app (`/picker/*`). Stack: React Router v6, TanStack React Query v5, Axios.

13.1 Backend access — the Vite proxy

Axios `baseUrl = /api`; `vite.config.ts` proxies `/api/*` → `http://localhost:8080/*` (stripping `/api`). This avoids **CORS** in dev and the **route↔API-path collision** (the SPA's own routes vs the backend's). Override with `VITE_API_BASE_URL` for a direct/CORS setup.

13.2 Single shared Axios instance + silent refresh (`src/api/http.ts`)

One client for both areas:

- Tokens in `localStorage` (`tokenStore`).
- **Request interceptor** attaches `Authorization: Bearer <access>`.
- **Response interceptor**: on **401** (non-auth call, not already retried) → one **silent refresh** via `POST /auth/refresh`, then replay the original request; concurrent 401s share one in-flight refresh; on failure → clear tokens + redirect to `/login`. This mirrors the backend's short-access/long-refresh token design — the access token can expire mid-session and the user never notices.

13.3 Routing

- `/login` — public.
- `RequireAuth` (auth-only outlet) wraps everything else:
 - `/` — `HomePage` (links to Admin Console + Picker App, logout).
 - `/admin/*` — `AdminLayout` (admin theme + drawer): Dashboard, Users (+ map-picker), Warehouses, Products, Shelf Mapping, Upload Orders (per-row report), Reports (filters + CSV/Excel download).
 - `/picker/*` — `PickerLayout` (nested `ThemeProvider` with a high-contrast `pickerTheme` per BRD §4 — WCAG AA, ≥44px touch targets for rugged devices): Dashboard, Select Warehouse, Claim, Picking Screen (current-step emphasis, simulate-scan, skip, wrong-item feedback), Completed.

Current limitation: `RequireAuth` is **auth-only**, not role-aware — both roles can currently reach both areas. Role-based route guarding is the documented next step. `AuthContext` resolves the session at boot via `/users/me`.

13.4 Pages → endpoints

- **Admin:** Dashboard (counts from `/users`, `/warehouses`, `/products`); Users (CRUD `/users` + `/map-picker`); Warehouses (`/warehouses`); Products (`/products`); Shelf Mapping (`/shelf-locations`, `/shelf-location`); Upload Orders (`/orders/upload`); Reports (`/report` JSON + CSV/Excel blob).
- **Picker:** Select Warehouse (`/picker/warehouses`, `/picker/select-warehouse`); Dashboard (`/picker/dashboard`, 409 → redirect to select-warehouse); Claim (`/orders/{id}/route` + `/orders/{id}/claim`); Picking (`/orders/{id}/route`, `/scan`, `/orders/{id}/items/{itemId}/skip`); Completed.

The frontend ships separately (not wired into the Maven build); `npm run build` is green.

14. Design Patterns Used

A catalogue of the patterns an interviewer will probe, with where each lives:

Pattern	Where	Why
Layered (n-tier) architecture	Controller → Service → Repository → Entity	Separation of concerns, testability.
Package-by-feature	auth , orders , picking , ...	Cohesion; everything about a feature in one folder.
DTO pattern	*Request / *Response records	Decouples the API contract from entities; prevents leaking fields (e.g. passwordHash).
Repository pattern	Spring Data JpaRepository interfaces	Abstracts persistence; derived queries + @Query .
Strategy	OrderFileParser (CSV vs Excel)	Swap parsing algorithm by file type without touching callers.
Factory	OrderFileParserFactory	Selects the right strategy by extension; Spring injects all strategies.
Static factory methods	ApiError.of , TokenResponse.bearer , OrderUploadRowResult.success/failed/skipped , ReportRow	Readable, intention-revealing construction of immutable records.
Observer / Publish-Subscribe	ApplicationEventPublisher + @TransactionalEventListener (audit)	Decouples auditing from business code; cross-cutting.
Optimistic locking	Order.@Version + the conditional claim UPDATE	Lock-free concurrency on order claiming.
Idempotency key	pick_logs.scan_id UNIQUE	Safe scanner retries.
Filter / Chain of Responsibility	Spring Security filter chain + JwtAuthenticationFilter	Pre-controller auth.
Front Controller	Spring MVC DispatcherServlet	Single entry point routing to controllers.
Centralized exception handling	@RestControllerAdvice	One uniform error contract.
Singleton	Spring-managed beans (default scope)	Single shared instances.
Dependency Injection	Constructor injection everywhere	Loose coupling, testability.
Transactional outbox	(not yet) — noted as the durability upgrade for audit	—

15. Interview Questions & Answers

A. Basics & architecture

Q1. Walk me through what happens when a request hits this app. The request passes the Spring Security filter chain first; `JwtAuthenticationFilter` verifies the Bearer JWT and populates the `SecurityContext` with an `AuthenticatedUser` principal and a `ROLE_*` authority. Then `@PreAuthorize` on the target controller method authorizes by role. The controller (thin) validates the `@Valid` body and calls a service. The service runs business rules inside a `@Transactional` boundary, using repositories (Spring Data JPA → SQL) to read/write Postgres, and may publish an `AuditEvent`. On commit, the response DTO is serialized to JSON. Exceptions are converted to a uniform `ApiError` by the `@RestControllerAdvice`.

Q2. Why package-by-feature instead of package-by-layer? Cohesion. Everything about "orders" (entity, repo, service, controller, DTOs) lives together, so you reason about and change a feature in one place. It scales better and makes ownership obvious as the codebase grows.

Q3. Why DTOs separate from entities? The API contract should not be the database schema. DTOs prevent leaking sensitive/internal fields (e.g. `password_hash`), let the DB evolve without breaking clients, and are where Bean Validation lives. Entities stay focused on persistence and lazy relationships that must not escape a transaction.

Q4. What does `@SpringBootApplication` do? It bundles `@SpringBootConfiguration`, `@EnableAutoConfiguration` (configures beans from the classpath — Tomcat, JPA, Security), and `@ComponentScan` (scans the package and below). Placed at the root package so scanning covers all features.

B. Persistence & JPA

Q5. Who owns the schema — Hibernate or SQL? SQL. `ddl-auto: validate` means Hibernate only *validates* mappings against the existing tables and never creates/alters them. The full schema + seed lives in the idempotent `db/schema.sql`, run on every boot by Spring SQL init (before JPA validates). Flyway is on the classpath but disabled.

Q6. Why `defer-datasource-initialization: false`? So the `schema.sql` runs *before* the JPA `EntityManagerFactory` is built. The tables must exist before Hibernate's `validate` inspects them; deferring would make `validate` fail on a fresh DB.

Q7. Why `VARCHAR + CHECK` for status columns instead of native PG enums? They map cleanly to JPA `@Enumerated(EnumType.STRING)`, are easy to extend (edit the CHECK, no `ALTER TYPE`), and are portable. `@Enumerated(STRING)` stores the name (not the ordinal) so reordering the Java enum can't corrupt data.

Q8. Why is `products.sku` the primary key but other tables use `BIGINT IDENTITY`? SKU is the real business identifier referenced directly by `shelf_locations`, `order_items`, `pick_logs` — a natural key. A consequence: with an *assigned* String id, Hibernate can't infer INSERT-vs-UPDATE from "id is null", so `ProductService.create` does an explicit `existsById` check before saving.

Q9. What's `open-in-view: false` and why? It disables Open-Session-In-View, so the Hibernate session doesn't stay open for the whole request/view rendering. Lazy loading is forced into the service transaction, which prevents accidental lazy queries during serialization and the associated performance surprises.

Q10. How are the picker↔warehouse composite keys modeled? `@EmbeddedId` with an `@Embeddable` `PickerWarehouseId` (the two-column key as a single field). The key class implements `Serializable` and overrides `equals` / `hashCode` (mandatory for JPA identity).

C. Security

Q11. Why JWT and why stateless? Stateless = no server session store, so the app scales horizontally without sticky sessions. The access token is a self-describing HS256 JWT carrying `sub/role/email/warehouseIds`, so the per-request hot path needs no DB lookup — the filter just verifies the signature.

Q12. If JWTs are stateless, how do you log out / revoke? The refresh token is **not** a JWT — it's an opaque 256-bit random string whose **SHA-256 hash** is stored in `refresh_tokens`. That gives a server-side revocation list: logout sets `revoked=true`; refresh rotates (revoke old, issue new); deleting a user wipes their tokens. Access tokens are short-lived (15 min) so the revocation window is small.

Q13. Why hash the refresh token with SHA-256, but passwords with BCrypt? The refresh token is already high-entropy (256 random bits) — nothing to brute-force — so a fast digest suffices. Passwords are low-entropy and human-chosen, so they need BCrypt's slow, salted, adaptive hashing to resist brute force.

Q14. How does RBAC work here? `@EnableMethodSecurity` + an explicit `@PreAuthorize` on **every** controller method. The filter adds a `ROLE_<role>` authority, so `hasRole('CENTRAL_ADMIN')` / `hasAnyRole( ... )` work. Public endpoints use `permitAll()`.

Q15. 401 vs 403 here? 401 (`RestAuthenticationEntryPoint`) = unauthenticated (no/invalid/expired token) — "who are you?". 403 (`RestAccessDeniedHandler`) = authenticated but wrong role — "you may not". Both emit the same `ApiError` shape, and the `GlobalExceptionHandler` mirrors them for failures raised during method invocation.

Q16. What is the `warehouseIds` claim for? Data isolation (BRD 3.1). For a `HUB_PICKER` it's their mapped warehouse ids (read at login); for `CENTRAL_ADMIN` it's empty = global. `WarehouseScope` derives every warehouse filter from this claim — a client-supplied warehouse id can narrow but never widen access; a foreign id → 403. Trade-off: the claim is fixed for the token's 15-min life; mapping changes take effect on next login/refresh.

Q17. Why is CSRF disabled? CSRF attacks ride on ambient credentials (cookies/session). This API is token-based and stateless — the token is sent explicitly in a header, never auto-attached by the browser — so CSRF doesn't apply.

D. Concurrency

Q18. Two pickers claim the same order at the same time — what happens? The claim is a single conditional UPDATE: `set status=ASSIGNED, picker_id=?, version=version+1 where id=? and status=PENDING and version=?`. The DB serializes the row update; exactly one matches (`updated==1` → wins), the other now sees a changed version/status (`updated==0` → 409 "already claimed"). Atomic, no race window, no exception needed for the common case.

Q19. Why the explicit UPDATE instead of loading the entity and relying on `@Version`? The conditional UPDATE is atomic at the DB (no Java read-modify-write window), it encodes the *business* precondition `status=PENDING` (not just version equality), and a lost race is a clean `updated==0` return value rather than a thrown `OptimisticLockException` to catch and translate. `clearAutomatically/flushAutomatically` keep the persistence context consistent around the bulk write.

Q20. How does scan idempotency work? Two layers. An optional client `scanId` is checked first (`findByScanId` → known ⇒ no-op replay with `duplicate=true`). The durable backstop is the `pick_logs.scan_id` UNIQUE constraint: a concurrent duplicate that races past the read fails the insert and rolls the scan back, so a pick can never be double-counted. NULL scan ids are distinct, so keyless scans still work. This matters because rugged scanners on flaky networks retry.

E. Cross-cutting

Q21. How is the audit trail implemented and why that way? Services publish an `AuditEvent`; an `@Async("auditExecutor") @TransactionalEventListener(AFTER_COMMIT)` writes `audit_logs` in a `REQUIRES_NEW` transaction on a separate thread. `AFTER_COMMIT` means only committed actions are audited (a rolled-back scan logs nothing); `@Async` means the latency-critical scan path never blocks on the audit write. One publish line per action; the write logic lives in one place.

Q22. What's the weakness of that audit design and how would you fix it? It's in-process async: between commit and the async write, the event lives only in the JVM's queue — a crash there loses it (no exactly-once guarantee). Fix with a **transactional outbox** (write an outbox row in the same transaction, a relay publishes it) or a durable broker (Kafka). Trade-off accepted here for low latency + simplicity.

Q23. How does the order upload stay resilient yet strict? File-level problems (bad/missing header, empty, unreadable, unsupported type) → 400, whole upload rejected. Once parsing succeeds, row failures never fail the batch (200 + per-row report). And each order is **atomic**: if any row in an `Order_ID` group fails or the group has inconsistent Customer/Warehouse, nothing for that order persists (bad rows FAILED, valid siblings SKIPPED) — no half-ingested orders.

Q24. Why normalize Excel numeric cells (`5.0` → `"5"`)? POI stores numbers as doubles, so an integer quantity reads as `5.0`. The validator does `Integer.parseInt`, which rejects `"5.0"`. Normalizing whole-number numerics to clean integer strings keeps a quantity of `5` valid whether it came from CSV or Excel.

Q25. Why build the report's JPQL dynamically instead of `:param is null or col = :param`? On PostgreSQL the static null-guard throws "could not determine data type of parameter" — PG can't infer a bind parameter's type when it may be NULL with no surrounding type context. Appending each clause only when its value is present binds the parameter only where its type is unambiguous (and produces a tighter plan). The report also uses a theta join to `Warehouse` because `Order` has no mapped `@ManyToOne` to it.

Q26. How does routing produce the right walking order? Items are sorted lexicographically by `location_code` (`Aisle_A-Bay_01-Shelf_1` ...) with unmapped items last. Because the numeric segments are zero-padded in the data (`Bay_04`, not `Bay_4`), string order equals travel order. The "current step" is the next PENDING item by that order, falling back to the first SKIPPED item so the picker is never blocked.

F. Stretch / senior

Q27. Where could N+1 query problems arise and how are they mitigated? Routing iterates an order's items and looks up each shelf location; `OrderItem.order` is LAZY. The repository offers a batch `findByWarehouseIdAndSkuIn` for SKU sets, and ingestion memoizes warehouse/shelf/order lookups in maps to avoid per-row queries. `open-in-view: false` also forces lazy loads into the service tx where they're visible.

Q28. How would you add "accept-short" order completion? Add `POST /orders/{id}/complete` that marks remaining SKIPPED items as short-accepted, completes the order, and records the shortfall (`Quantity_Picked < Quantity_Ordered`) so the dispatch report reflects it. Today an order completes only when every item is PICKED, so a permanently out-of-stock SKIPPED item keeps it open.

Q29. How is testing structured? Full-context Spring integration tests against a real Postgres (`@Transactional` rollback for most; the concurrency test commits across two threads and cleans up explicitly). Coverage includes RBAC (401/403/200), data isolation, product/location mapping, order ingestion, picker workflow, scan-to-pick (incl. idempotency), claim concurrency, audit async, and reporting.

Q30. Walk the complete lifecycle of one order. Admin uploads a file → atomic `PENDING` order + items created (each SKU validated to have a shelf in the warehouse). Picker selects their active warehouse → dashboard shows the order as *available* → claims it (optimistic lock → `ASSIGNED`) → opens the route (sorted by location) → scans items (`PICKING`; each scan increments, logs, current-step enforced; idempotent on `scanId`; skip/revisit for blocked items) → last item `PICKED` flips the order to `COMPLETED` and closes the session. Every step publishes audit events written after commit. Admin pulls the dispatch report (fulfillment rate per line) and the access is itself audited.

16. Improvements & Best Practices

Things already done well and concrete next steps — useful to volunteer in an interview.

Already strong: stateless JWT with revocable hashed refresh tokens; explicit `@PreAuthorize` on every method; SQL-owned schema with Hibernate `validate`; uniform error contract; off-thread auditing that respects transaction boundaries; lock-free optimistic claiming; idempotent scans; clean Strategy/Factory parsing; constructor injection throughout; `open-in-view: false`.

Recommended improvements:

1. **Harden audit durability** — move to a transactional outbox or a durable broker so audit events survive a JVM crash between commit and write (exactly-once).
 2. **Role-based route guards on the frontend** — `RequireAuth` is auth-only today; both roles can reach both areas. Add role gating.
 3. **Refresh-token housekeeping** — a scheduled job to purge expired/revoked `refresh_tokens` (`@Scheduled`); there's no cleanup today.
 4. **Externalize all secrets** — the dev JWT secret is committed (overridable via env). Enforce env-only secrets in non-dev (and rotate). Likewise DB passwords.
 5. **Validation depth** — add `@Positive` on quantities at the DTO level, and consider a per-field error map in the validation handler (it currently returns only the first field error).
 6. **Pagination** — `GET /users`, `/products`, `/report`, dashboards return full lists; add `Pageable` / `Slice` for scale.
 7. **Accept-short completion** (Q28) and **completion-date reporting** (use `pick_sessions.end_time` rather than order creation date) — both documented enhancements.
 8. **Refresh the `warehouseIds` claim** when a picker's mappings change mid-session (today it propagates only on next login/refresh) — or shorten the access-token TTL further.
 9. **Observability** — add Spring Boot Actuator health/metrics, structured request logging, and a correlation id; tighten the catch-all 500 with logging of the stack (server-side only).
 10. **Testcontainers** — pin integration tests to a disposable Postgres container in CI so they never pollute or depend on a shared DB.
 11. **Rate limiting / logout** on `/auth/login` to resist credential stuffing.
 12. **MapStruct mappers** — the build is wired for MapStruct (`spring` component model) but most mapping is hand-written `from( ... )` factories; either adopt MapStruct consistently or drop the dependency to reduce build surface.
-

End of guide. Generated from a full read of the backend source, the SQL schema, configuration, and the project design docs.